<PROJECT PDF DUMP>
Generated at: 2026-04-17T19:03:01.080683 UTC
Root: C:\Users\morty\video-exchange-bot\app
Files included: 12

==
FILE: __init__.py
==

==
FILE: admin_handlers.py
==
```
import traceback
from datetime import datetime

from aiogram import Router, F
from aiogram.filters import Command
from aiogram.fsm.state import State, StatesGroup
from aiogram.fsm.context import FSMContext
from aiogram.types import Message, CallbackQuery

from app.config import ADMINS, LOG_CHAT_ID
from app.db import async_session
from app.services import (
    get_user,
    get_next_pending_video,
    approve_video,
    reject_video,
    approve_all_pending,
    count_pending_videos,
    count_approved_videos,
    count_rejected_videos,
    format_duration,
    format_file_size,
    get_user_by_id,
    create_offer,
    get_all_offers,
    toggle_offer_active,
    get_user_by_username,
    set_user_admin,
    get_db_admins,
    get_admin_extended_stats,
)
from app.keyboards import (
    moderation_keyboard,
    rejection_reason_keyboard,
    admin_center_keyboard,
    admin_after_action_keyboard,
    admin_offers_menu_keyboard,
    admin_offer_list_keyboard,
    admin_manage_keyboard,
    back_to_admin_manage_keyboard,
)
from app.logger import get_logger, log_info, log_warning, log_exception

logger = get_logger(__name__)
router = Router()

REASON_MAP = {
    "duplicate": "\u0414\u0443\u0431\u043b\u0438\u043a\u0430\u0442",
    "off_topic": "\u041d\u0435 \u043f\u043e \u0442\u0435\u043c\u0430\u0442\u0438\u043a\u0435",
    "other": "\u0414\u0440\u0443\u0433\u043e\u0435",
}


class OfferCreateState(StatesGroup):
    title = State()
    description = State()
    channel_url = State()
```

```
class AdminManageState(StatesGroup):
    waiting_add_username = State()
    waiting_remove_username = State()


def is_super_admin(tid: int) -> bool:
    return tid in ADMINS


async def check_admin(tid: int) -> bool:
    if tid in ADMINS:
        return True

    async with async_session() as session:
        user = await get_user(session, tid)
        if user and user.is_admin:
            return True

    return False


async def send_admin_log(bot, text: str):
    if not LOG_CHAT_ID:
        return

    try:
        await bot.send_message(int(LOG_CHAT_ID), text, parse_mode="HTML")
    except Exception as e:
        log_warning(
            logger,
            "■■ ■■■■■■■■ ■■■■■■■■■■ ■■■■ ■ Telegram",
            error=str(e),
            log_chat_id=LOG_CHAT_ID,
        )


@router.message(Command("admin"))
async def cmd_admin(message: Message):
    if not message.from_user:
        return

    ok = await check_admin(message.from_user.id)
    if not ok:
        await message.answer("\u26d4")
        return

    sa = is_super_admin(message.from_user.id)

    log_info(
        logger,
        "■■■■■■■■ ■■■■■■-■■■■■■■ ■■■■■■ ■■■■■■■■ /admin",
        tg_id=message.from_user.id,
        is_super_admin=sa,
    )

    await message.answer(
        "\U0001f6e0 <b>\u0410\u0434\u043c\u0438\u043d</b>",
        parse_mode="HTML",
        reply_markup=admin_center_keyboard(is_super_admin=sa),
    )


@router.callback_query(F.data == "admin_queue_info")
async def cb_queue(callback: CallbackQuery):
    if not callback.from_user:
        return

    ok = await check_admin(callback.from_user.id)
```

```
    if not ok:
        await callback.answer("\u26d4", show_alert=True)
        return

    try:
        async with async_session() as session:
            p = await count_pending_videos(session)
            a = await count_approved_videos(session)
            r = await count_rejected_videos(session)

        sa = is_super_admin(callback.from_user.id)

        text = (
            f"\U0001f4ca <b>\u0421\u0442\u0430\u0442\u0438\u0441\u0442\u0438\u043a\u0430</b>\n\n
"
            f"\u23f3 \u041d\u0430 \u043c\u043e\u0434\u0435\u0440\u0430\u0446\u0438\u0438: <b>{p}
</b>\n"
            f"\u2705 \u041e\u0434\u043e\u0431\u0440\u0435\u043d\u043e: <b>{a}</b>\n"
            f"\u274c \u041e\u0442\u043a\u043b\u043e\u043d\u0435\u043d\u043e: <b>{r}</b>"
        )

        log_info(
            logger,
            "[redacted]",
            tg_id=callback.from_user.id,
            pending=p,
            approved=a,
            rejected=r,
        )

        try:
            await callback.message.edit_text(
                text,
                parse_mode="HTML",
                reply_markup=admin_center_keyboard(is_super_admin=sa),
            )
        except Exception:
            await callback.message.answer(
                text,
                parse_mode="HTML",
                reply_markup=admin_center_keyboard(is_super_admin=sa),
            )

        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[redacted]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data == "admin_extended_stats")
async def cb_extended_stats(callback: CallbackQuery):
    if not callback.from_user:
        return

    ok = await check_admin(callback.from_user.id)
    if not ok:
        await callback.answer("\u26d4", show_alert=True)
        return

    try:
        async with async_session() as session:
            stats = await get_admin_extended_stats(session)

        text = (
            "\U0001f4ca <b>\u0421\u0442\u0430\u0442\u0438\u0441\u0442\u0438\u043a\u0430+</b>\n\n
"
```

```
            f"\U0001f465 \u041f\u043e\u043b\u044c\u0437\u043e\u0432\u0430\u0442\u0435\u043b\u0438
8: <b>{stats['users']}</b>\n"
            f"\U0001f48e VIP: <b>{stats['vip']}</b>\n"
            f"\U0001f4ac \u041a\u043e\u043c\u043c\u0435\u043d\u0442\u044b: <b>{stats['comments']
}</b>\n"
            f"\u2764\ufe0f \u0420\u0435\u0430\u043a\u0446\u0438\u0438: <b>{stats['reactions']}</
b>\n"
            f"\U0001f3ae \u0418\u0433\u0440\u044b: <b>{stats['games']}</b>\n"
            f"\U0001f381 \u041e\u0444\u0444\u0435\u0440\u044b: <b>{stats['offers']}</b>"
        )

        log_info(
            logger,
            "[redacted]",
            tg_id=callback.from_user.id,
            **stats,
        )

        await callback.message.answer(text, parse_mode="HTML")
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[redacted]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data == "admin_logs")
async def cb_admin_logs(callback: CallbackQuery):
    if not callback.from_user:
        return

    if not await check_admin(callback.from_user.id):
        await callback.answer("\u26d4", show_alert=True)
        return

    if not LOG_CHAT_ID:
        await callback.answer("LOG_CHAT_ID \u043d\u0435 \u0437\u0430\u0434\u0430\u043d", show_al
ert=True)
        return

    log_info(
        logger,
        "[redacted]",
        tg_id=callback.from_user.id,
        log_chat_id=LOG_CHAT_ID,
    )

    text = (
        "\U0001f4dc <b>\u041b\u043e\u0433-\u0446\u0435\u043d\u0442\u0440</b>\n\n"
        f"\u041b\u043e\u0433\u0438 \u0438\u0434\u0443\u0442 \u0432 chat_id: <code>{LOG_CHAT_ID}<
/code>\n"
        "\u0412\u0430\u0436\u043d\u044b\u0435 \u0441\u043e\u0431\u044b\u0442\u0438\u044f \u0431\
u0443\u0434\u0443\u0442 \u043f\u0440\u0438\u0445\u043e\u0434\u0438\u0442\u044c \u0442\u0443\u043
4\u0430."
    )
    await callback.message.answer(text, parse_mode="HTML")
    await callback.answer()


@router.callback_query(F.data == "admin_approve_all")
async def cb_approve_all(callback: CallbackQuery):
    if not callback.from_user:
        return

    ok = await check_admin(callback.from_user.id)
    if not ok:
        await callback.answer("\u26d4", show_alert=True)
```

```
        return

    try:
        async with async_session() as session:
            count = await approve_all_pending(session)

        sa = is_super_admin(callback.from_user.id)
        text = f"\u2705 \u041e\u0434\u043e\u0431\u0440\u0435\u043d\u043e \u0432\u0441\u0451: <b>
{count}</b> \u0435\u0434."

        log_info(
            logger,
            "[redacted] [redacted] [redacted] [redacted]",
            tg_id=callback.from_user.id,
            approved_count=count,
        )

        await callback.message.answer(
            text,
            parse_mode="HTML",
            reply_markup=admin_center_keyboard(is_super_admin=sa),
        )

        await send_admin_log(
            callback.bot,
            f"\U0001f7e2 <b>APPROVE ALL</b>\n"
            f"\u0410\u0434\u043c\u0438\u043d: <code>{callback.from_user.id}</code>\n"
            f"\u041e\u0434\u043e\u0431\u0440\u0435\u043d\u043e: <b>{count}</b>\n"
            f"\u0412\u0440\u0435\u043c\u044f: {datetime.utcnow().strftime('%Y-%m-%d %H:%M:%S')}
UTC"
        )

        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[redacted] [redacted] [redacted] [redacted] [redacted]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data == "admin_get_pending")
async def cb_pending(callback: CallbackQuery):
    if not callback.from_user:
        return

    ok = await check_admin(callback.from_user.id)
    if not ok:
        await callback.answer("\u26d4", show_alert=True)
        return

    log_info(
        logger,
        "[redacted] [redacted] pending-[redacted]",
        tg_id=callback.from_user.id,
    )

    await callback.answer()
    await _send_pending(callback.message, callback.from_user.id)


async def _send_pending(message, admin_id: int | None = None):
    try:
        async with async_session() as session:
            pc = await count_pending_videos(session)
            video = await get_next_pending_video(session)

            if not video:
                await message.answer(
```

```
                    "\u2705 \u041f\u0443\u0441\u0442\u043e.",
                    reply_markup=admin_center_keyboard(
                        is_super_admin=admin_id in ADMINS if admin_id else False
                    ),
                )
                return

            fid = video.telegram_file_id
            vid = video.id
            uid = video.uploader_user_id
            ct = video.content_type
            dur = format_duration(video.duration_seconds) if ct == "video" else "\u2014"
            sz = format_file_size(video.file_size)
            label = "\U0001f5bc \u0424\u043e\u0442\u043e" if ct == "photo" else "\U0001f3ac \u04
12\u0438\u0434\u0435\u043e"

            cap = (
                f"\U0001f4cb <b>\u041c\u043e\u0434\u0435\u0440\u0430\u0446\u0438\u044f</b>\n\n"
                f"{label} #{vid}\n"
                f"\u0410\u0432\u0442\u043e\u0440: {uid}\n"
                f"\u0414\u043b\u0438\u0442.: {dur}\n"
                f"\u0420\u0430\u0437\u043c.: {sz}\n"
                f"\u041e\u0447\u0435\u0440\u0435\u0434\u044c: {pc}"
            )

            log_info(
                logger,
                "[redacted] pending-[redacted]",
                admin_id=admin_id,
                video_id=vid,
                uploader_id=uid,
                content_type=ct,
                pending_total=pc,
            )

            if ct == "photo":
                await message.answer_photo(
                    photo=fid,
                    caption=cap,
                    parse_mode="HTML",
                    reply_markup=moderation_keyboard(vid),
                )
            else:
                await message.answer_video(
                    video=fid,
                    caption=cap,
                    parse_mode="HTML",
                    reply_markup=moderation_keyboard(vid),
                )
    except Exception:
        log_exception(
            logger,
            "[redacted] pending-[redacted]",
            admin_id=admin_id,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


@router.callback_query(F.data.startswith("mod_approve:"))
async def cb_approve(callback: CallbackQuery):
    if not callback.from_user:
        return

    ok = await check_admin(callback.from_user.id)
    if not ok:
        await callback.answer("\u26d4", show_alert=True)
        return

    try:
        vid = int(callback.data.split(":")[1])
```

```
        async with async_session() as session:
            video = await approve_video(session, vid)
            uploader = await get_user_by_id(session, video.uploader_user_id) if video else None

            if not video:
                await callback.message.edit_caption(
                    caption=f"#{vid} \u043d\u0435 \u043d\u0430\u0439\u0434\u0435\u043d.",
                    reply_markup=admin_after_action_keyboard(),
                )
                await callback.answer()
                return

            rw = "0.1" if video.content_type == "photo" else "0.5"

            log_info(
                logger,
                "[redacted]",
                tg_id=callback.from_user.id,
                video_id=vid,
                content_type=video.content_type,
                reward=rw,
                uploader_id=video.uploader_user_id,
            )

            await callback.message.edit_caption(
                caption=f"\u2705 #{vid} +{rw}",
                reply_markup=admin_after_action_keyboard(),
            )

            if uploader:
                try:
                    await callback.bot.send_message(
                        uploader.telegram_id,
                        f"\u2705 #{vid} \u043e\u0434\u043e\u0431\u0440\u0435\u043d. +<b>{rw}</b>
",
                        parse_mode="HTML",
                    )
                except Exception:
                    pass

            await send_admin_log(
                callback.bot,
                f"\U0001f7e2 <b>APPROVED</b>\n"
                f"\u0410\u0434\u043c\u0438\u043d: <code>{callback.from_user.id}</code>\n"
                f"\u041a\u043e\u043d\u0442\u0435\u043d\u0442: <b>#{vid}</b>\n"
                f"\u0422\u0438\u043f: {video.content_type}\n"
                f"\u0410\u0432\u0442\u043e\u0440: <code>{video.uploader_user_id}</code>"
            )

            await callback.answer("\u041e\u0434\u043e\u0431\u0440\u0435\u043d\u043e")
        except Exception:
            log_exception(
                logger,
                "[redacted]",
                tg_id=callback.from_user.id if callback.from_user else None,
            )
            await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


    @router.callback_query(F.data.startswith("mod_reject:"))
    async def cb_reject(callback: CallbackQuery):
        if not callback.from_user:
            return

        ok = await check_admin(callback.from_user.id)
        if not ok:
            await callback.answer("\u26d4", show_alert=True)
            return
```

```
    vid = int(callback.data.split(":")[1])

    log_info(
        logger,
        "[redacted]",
        tg_id=callback.from_user.id,
        video_id=vid,
    )

    await callback.message.edit_reply_markup(reply_markup=rejection_reason_keyboard(vid))
    await callback.answer()


@router.callback_query(F.data.startswith("reject_reason:"))
async def cb_reject_reason(callback: CallbackQuery):
    if not callback.from_user:
        return

    ok = await check_admin(callback.from_user.id)
    if not ok:
        await callback.answer("\u26d4", show_alert=True)
        return

    try:
        parts = callback.data.split(":", 2)
        vid = int(parts[1])
        rk = parts[2]
        rt = REASON_MAP.get(rk, "\u0414\u0440\u0443\u0433\u043e\u0435")

        async with async_session() as session:
            video = await reject_video(session, vid, rt)
            uploader = await get_user_by_id(session, video.uploader_user_id) if video else None

            if not video:
                await callback.message.edit_caption(
                    caption=f"#{vid} \u043d\u0435 \u043d\u0430\u0439\u0434\u0435\u043d.",
                    reply_markup=admin_after_action_keyboard(),
                )
                await callback.answer()
                return

            log_info(
                logger,
                "[redacted]",
                tg_id=callback.from_user.id,
                video_id=vid,
                reason=rt,
                uploader_id=video.uploader_user_id,
            )

            await callback.message.edit_caption(
                caption=f"\u274c #{vid}: {rt}",
                reply_markup=admin_after_action_keyboard(),
            )

            if uploader:
                try:
                    await callback.bot.send_message(
                        uploader.telegram_id,
                        f"\u274c #{vid}: <b>{rt}</b>",
                        parse_mode="HTML",
                    )
                except Exception:
                    pass

            await send_admin_log(
                callback.bot,
                f"\U0001f534 <b>REJECTED</b>\n"
                f"\u0410\u0434\u043c\u0438\u043d: <code>{callback.from_user.id}</code>\n"
                f"\u041a\u043e\u043d\u0442\u0435\u043d\u0442: <b>#{vid}</b>\n"
```

```
                f"\u041f\u0440\u0438\u0447\u0438\u043d\u0430: <b>{rt}</b>\n"
                f"\u0410\u0432\u0442\u043e\u0440: <code>{video.uploader_user_id}</code>"
            )

            await callback.answer("\u041e\u0442\u043a\u043b\u043e\u043d\u0435\u043d\u043e")
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data == "admin_offers_menu")
async def offers_menu(callback: CallbackQuery):
    if not callback.from_user:
        return

    ok = await check_admin(callback.from_user.id)
    if not ok:
        await callback.answer("\u26d4", show_alert=True)
        return

    log_info(
        logger,
        "[illegible]",
        tg_id=callback.from_user.id,
    )

    await callback.message.answer(
        "\U0001f381 <b>\u041e\u0444\u0444\u0435\u0440\u044b</b>",
        parse_mode="HTML",
        reply_markup=admin_offers_menu_keyboard(),
    )
    await callback.answer()


@router.callback_query(F.data == "admin_offer_create")
async def offer_create(callback: CallbackQuery, state: FSMContext):
    if not callback.from_user:
        return

    ok = await check_admin(callback.from_user.id)
    if not ok:
        await callback.answer("\u26d4", show_alert=True)
        return

    log_info(
        logger,
        "[illegible]",
        tg_id=callback.from_user.id,
    )

    await state.set_state(OfferCreateState.title)
    await callback.message.answer("\u041d\u0430\u0437\u0432\u0430\u043d\u0438\u0435 \u043e\u0444
\u0444\u0435\u0440\u0430:")
    await callback.answer()


@router.message(OfferCreateState.title)
async def oc_title(message: Message, state: FSMContext):
    await state.update_data(title=message.text or "")
    await state.set_state(OfferCreateState.description)
    await message.answer("\u041e\u043f\u0438\u0441\u0430\u043d\u0438\u0435:")


@router.message(OfferCreateState.description)
async def oc_desc(message: Message, state: FSMContext):
    await state.update_data(description=message.text or "")
```

```
    await state.set_state(OfferCreateState.channel_url)
    await message.answer("\u0421\u0441\u044b\u043b\u043a\u0430 \u043d\u0430 \u043a\u0430\u043d\u0430\u043b:")


@router.message(OfferCreateState.channel_url)
async def oc_url(message: Message, state: FSMContext):
    data = await state.get_data()

    try:
        async with async_session() as session:
            offer = await create_offer(
                session,
                data["title"],
                data["description"],
                message.text or "",
            )

        log_info(
            logger,
            "[illegible]",
            tg_id=message.from_user.id if message.from_user else None,
            offer_id=offer.id,
            title=offer.title,
        )

        await message.answer(
            f"\u2705 \u041e\u0444\u0444\u0435\u0440 #{offer.id} \u0441\u043e\u0437\u0434\u0430\u043d.",
            reply_markup=admin_offers_menu_keyboard(),
        )

        await send_admin_log(
            message.bot,
            f"\U0001f381 <b>NEW OFFER</b>\n"
            f"\u0410\u0434\u043c\u0438\u043d: <code>{message.from_user.id}</code>\n"
            f"ID: <b>{offer.id}</b>\n"
            f"\u041d\u0430\u0437\u0432\u0430\u043d\u0438\u0435: {offer.title}"
        )
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")
    finally:
        await state.clear()


@router.callback_query(F.data == "admin_offer_list")
async def offer_list(callback: CallbackQuery):
    if not callback.from_user:
        return

    ok = await check_admin(callback.from_user.id)
    if not ok:
        await callback.answer("\u26d4", show_alert=True)
        return

    try:
        async with async_session() as session:
            offers = await get_all_offers(session)

        log_info(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id,
            offers_count=len(offers),
        )
```

```
        if not offers:
            await callback.message.answer(
                "\u041d\u0435\u0442 \u043e\u0444\u0444\u0435\u0440\u043e\u0432.",
                reply_markup=admin_offers_menu_keyboard(),
            )
        else:
            await callback.message.answer(
                "\U0001f4cb",
                reply_markup=admin_offer_list_keyboard(offers),
            )

        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[redacted]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data.startswith("admin_offer_toggle:"))
async def offer_toggle(callback: CallbackQuery):
    if not callback.from_user:
        return

    ok = await check_admin(callback.from_user.id)
    if not ok:
        await callback.answer("\u26d4", show_alert=True)
        return

    try:
        oid = int(callback.data.split(":")[1])

        async with async_session() as session:
            offer = await toggle_offer_active(session, oid)

        if not offer:
            await callback.answer("\u041d\u0435 \u043d\u0430\u0439\u0434\u0435\u043d", show_aler
t=True)
            return

        st = "\u0432\u043a\u043b" if offer.is_active else "\u0432\u044b\u043a\u043b"

        log_info(
            logger,
            "[redacted]",
            tg_id=callback.from_user.id,
            offer_id=offer.id,
            is_active=offer.is_active,
        )

        await callback.message.answer(
            f"{offer.title} \u2014 {st}",
            reply_markup=admin_offers_menu_keyboard(),
        )

        await send_admin_log(
            callback.bot,
            f"\U0001f4cc <b>OFFER TOGGLE</b>\n"
            f"\u0410\u0434\u043c\u0438\u043d: <code>{callback.from_user.id}</code>\n"
            f"ID: <b>{offer.id}</b>\n"
            f"\u0421\u0442\u0430\u0442\u0443\u0441: <b>{st}</b>"
        )

        await callback.answer()
    except Exception:
        log_exception(
            logger,
```

```
            "[redacted]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data == "admin_manage_admins")
async def cb_manage(callback: CallbackQuery):
    if not callback.from_user or not is_super_admin(callback.from_user.id):
        await callback.answer("\u26d4", show_alert=True)
        return

    log_info(
        logger,
        "[redacted]",
        tg_id=callback.from_user.id,
    )

    await callback.message.edit_text(
        "\U0001f451 <b>\u0423\u043f\u0440\u0430\u0432\u043b\u0435\u043d\u0438\u0435 \u0430\u0434
\u043c\u0438\u043d\u0430\u043c\u0438</b>",
        parse_mode="HTML",
        reply_markup=admin_manage_keyboard(),
    )


@router.callback_query(F.data == "admm_list")
async def cb_list(callback: CallbackQuery):
    if not callback.from_user or not is_super_admin(callback.from_user.id):
        await callback.answer("\u26d4", show_alert=True)
        return

    try:
        async with async_session() as session:
            db_admins = await get_db_admins(session)

            lines = [
                "\U0001f451 <b>\u0410\u0434\u043c\u0438\u043d\u044b</b>\n",
                "<b>\U0001f534 \u0421\u0443\u043f\u0435\u0440 (env):</b>",
            ]

            for tid in ADMINS:
                u = await get_user(session, tid)
                if u:
                    lines.append(f" \u2022 @{u.username or u.telegram_id} ({u.first_name or ''})
")
                else:
                    lines.append(f" \u2022 {tid} (\u043d\u0435 \u0432 \u0411\u0414)")

            lines.append("\n<b>\U0001f7e2 \u041e\u0431\u044b\u0447\u043d\u044b\u0435 (\u0411\u04
14):</b>")
            non = [a for a in db_admins if a.telegram_id not in ADMINS]

            if non:
                for a in non:
                    lines.append(f" \u2022 @{a.username or a.telegram_id} ({a.first_name or ''})
")
            else:
                lines.append(" (\u043f\u0443\u0441\u0442\u043e)")

            log_info(
                logger,
                "[redacted]",
                tg_id=callback.from_user.id,
                env_admins=len(ADMINS),
                db_admins=len(db_admins),
            )

            await callback.message.edit_text(
                "\n".join(lines),
```

```
                parse_mode="HTML",
                reply_markup=back_to_admin_manage_keyboard(),
            )
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data == "admm_add")
async def cb_add(callback: CallbackQuery, state: FSMContext):
    if not callback.from_user or not is_super_admin(callback.from_user.id):
        await callback.answer("\u26d4", show_alert=True)
        return

    log_info(
        logger,
        "[illegible]",
        tg_id=callback.from_user.id,
    )

    await state.set_state(AdminManageState.waiting_add_username)
    await callback.message.answer(
        "\u2795 Username \u0434\u043b\u044f \u0434\u043e\u0431\u0430\u0432\u043b\u0435\u043d\u04
38\u044f (@user \u0438\u043b\u0438 user).\n/start \u0434\u043b\u044f \u043e\u0442\u043c\u0435\u0
43d\u044b."
    )
    await callback.answer()


@router.message(AdminManageState.waiting_add_username)
async def add_proc(message: Message, state: FSMContext):
    if not message.from_user or not is_super_admin(message.from_user.id):
        await state.clear()
        return

    inp = (message.text or "").strip()

    try:
        async with async_session() as session:
            user = await get_user_by_username(session, inp)
            if not user:
                await message.answer(f"\u274c {inp} \u043d\u0435 \u043d\u0430\u0439\u0434\u0435\
u043d.")
                await state.clear()
                return

            if user.telegram_id in ADMINS:
                await message.answer("\u26a0\ufe0f \u0423\u0436\u0435 \u0441\u0443\u043f\u0435\u0
440-\u0430\u0434\u043c\u0438\u043d.")
                await state.clear()
                return

            if user.is_admin:
                await message.answer("\u26a0\ufe0f \u0423\u0436\u0435 \u0430\u0434\u043c\u0438\u0
043d.")
                await state.clear()
                return

            await set_user_admin(session, user, True)

        log_info(
            logger,
            "[illegible]",
            initiator_tg_id=message.from_user.id,
            added_admin_tg_id=user.telegram_id,
            username=user.username,
```

```
        )

        await message.answer(f"\u2705 @{user.username} \u0442\u0435\u043f\u0435\u0440\u044c \u04
30\u0434\u043c\u0438\u043d!")

        await send_admin_log(
            message.bot,
            f"\U0001f7e2 <b>ADMIN ADDED</b>\n"
            f"\u0418\u043d\u0438\u0446\u0438\u0430\u0442\u043e\u0440: <code>{message.from_user.i
d}</code>\n"
            f"\u041d\u043e\u0432\u044b\u0439 \u0430\u0434\u043c\u0438\u043d: @{user.username or
user.telegram_id}"
        )
    except Exception:
        log_exception(
            logger,
            "███████ ███ ███████████ ████████████████",
            tg_id=message.from_user.id if message.from_user else None,
            input_username=inp,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")
    finally:
        await state.clear()


@router.callback_query(F.data == "admm_remove")
async def cb_rem(callback: CallbackQuery, state: FSMContext):
    if not callback.from_user or not is_super_admin(callback.from_user.id):
        await callback.answer("\u26d4", show_alert=True)
        return

    log_info(
        logger,
        "███████ █████████ ████████████████",
        tg_id=callback.from_user.id,
    )

    await state.set_state(AdminManageState.waiting_remove_username)
    await callback.message.answer(
        "\u2796 Username \u0434\u043b\u044f \u0443\u0434\u0430\u043b\u0435\u043d\u0438\u044f.\n/
start \u0434\u043b\u044f \u043e\u0442\u043c\u0435\u043d\u044b."
    )
    await callback.answer()


@router.message(AdminManageState.waiting_remove_username)
async def rem_proc(message: Message, state: FSMContext):
    if not message.from_user or not is_super_admin(message.from_user.id):
        await state.clear()
        return

    inp = (message.text or "").strip()

    try:
        async with async_session() as session:
            user = await get_user_by_username(session, inp)
            if not user:
                await message.answer(f"\u274c {inp} \u043d\u0435 \u043d\u0430\u0439\u0434\u0435\
u043d.")
                await state.clear()
                return

            if user.telegram_id in ADMINS:
                await message.answer("\u26d4 \u0421\u0443\u043f\u0435\u0440-\u0430\u0434\u043c\u
0438\u043d. \u041d\u0435\u043b\u044c\u0437\u044f.")
                await state.clear()
                return

            if not user.is_admin:
                await message.answer("\u26a0\ufe0f \u041d\u0435 \u0430\u0434\u043c\u0438\u043d."
```

```
)
                await state.clear()
                return

            await set_user_admin(session, user, False)

        log_info(
            logger,
            "[redacted]",
            initiator_tg_id=message.from_user.id,
            removed_admin_tg_id=user.telegram_id,
            username=user.username,
        )

        await message.answer(f"\u2705 @{user.username} \u0443\u0434\u0430\u043b\u0451\u043d.")

        await send_admin_log(
            message.bot,
            f"\U0001f534 <b>ADMIN REMOVED</b>\n"
            f"\u0418\u043d\u0438\u0446\u0438\u0430\u0442\u043e\u0440: <code>{message.from_user.i
d}</code>\n"
            f"\u0423\u0434\u0430\u043b\u0451\u043d\u043d\u044b\u0439 \u0430\u0434\u043c\u0438\u0
43d: @{user.username or user.telegram_id}"
        )
    except Exception:
        log_exception(
            logger,
            "[redacted]",
            tg_id=message.from_user.id if message.from_user else None,
            input_username=inp,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")
    finally:
        await state.clear()

================================================================================
FILE: config.py
================================================================================
import os
from dotenv import load_dotenv

load_dotenv()


def _get_str(name: str, default: str = "") -> str:
    return os.getenv(name, default).strip()


def _get_int(name: str, default: int) -> int:
    raw = os.getenv(name)
    if raw is None:
        return default
    raw = raw.strip()
    try:
        return int(raw)
    except Exception:
        return default


def _get_float(name: str, default: float) -> float:
    raw = os.getenv(name)
    if raw is None:
        return default
    raw = raw.strip()
    try:
        return float(raw)
    except Exception:
        return default
```

```
BOT_TOKEN = _get_str("BOT_TOKEN")
DATABASE_URL = _get_str("DATABASE_URL")
BOT_USERNAME = _get_str("BOT_USERNAME")
PROVIDER_TOKEN = _get_str("PROVIDER_TOKEN")
WEBHOOK_BASE = _get_str("WEBHOOK_BASE")
WEBHOOK_PATH = _get_str("WEBHOOK_PATH")
LOG_CHAT_ID = _get_str("LOG_CHAT_ID")

ADMINS_RAW = _get_str("ADMINS")
ADMINS = []
for x in ADMINS_RAW.split(","):
    x = x.strip()
    if x.isdigit():
        ADMINS.append(int(x))

PORT = _get_int("PORT", 10000)

STARTING_BALANCE = _get_float("STARTING_BALANCE", 2.0)
WATCH_COST = _get_float("WATCH_COST", 1.0)
UPLOAD_REWARD = _get_float("UPLOAD_REWARD", 0.5)

REFERRAL_REWARD_INVITER = _get_float("REFERRAL_REWARD_INVITER", 2.0)
REFERRAL_REWARD_NEW_USER = _get_float("REFERRAL_REWARD_NEW_USER", 1.0)

STARS_TO_COINS_RATE = _get_float("STARS_TO_COINS_RATE", 2.0)

STARS_PACKAGES = {
    "pack_1": {"stars": 1, "coins": 2, "title": "2 \u043c\u043e\u043d\u0435\u0442\u044b"},
    "pack_5": {"stars": 5, "coins": 10, "title": "10 \u043c\u043e\u043d\u0435\u0442"},
    "pack_10": {"stars": 10, "coins": 20, "title": "20 \u043c\u043e\u043d\u0435\u0442"},
    "pack_25": {"stars": 25, "coins": 50, "title": "50 \u043c\u043e\u043d\u0435\u0442"},
    "pack_50": {"stars": 50, "coins": 100, "title": "100 \u043c\u043e\u043d\u0435\u0442"},
}

MONEY_PACKAGES = {
    "money_99": {"amount": 9900, "coins": 250, "title": "250 \u043c\u043e\u043d\u0435\u0442"},
    "money_199": {"amount": 19900, "coins": 600, "title": "600 \u043c\u043e\u043d\u0435\u0442"},
    "money_499": {"amount": 49900, "coins": 1800, "title": "1800 \u043c\u043e\u043d\u0435\u0442"
},
}

OFFER_BROADCAST_INTERVAL_HOURS = _get_float("OFFER_BROADCAST_INTERVAL_HOURS", 2.5)

# === LEVELS ===
LEVEL_XP_BASE = _get_int("LEVEL_XP_BASE", 100)
LEVEL_XP_MULTIPLIER = _get_float("LEVEL_XP_MULTIPLIER", 1.5)

XP_PER_WATCH = _get_int("XP_PER_WATCH", 5)
XP_PER_UPLOAD = _get_int("XP_PER_UPLOAD", 20)
XP_PER_RATING = _get_int("XP_PER_RATING", 2)
XP_PER_COMMENT = _get_int("XP_PER_COMMENT", 3)
XP_PER_REACTION = _get_int("XP_PER_REACTION", 1)
XP_PER_GAME = _get_int("XP_PER_GAME", 3)

# === VIP ===
VIP_PRICE_STARS = _get_int("VIP_PRICE_STARS", 50)
VIP_DURATION_DAYS = _get_int("VIP_DURATION_DAYS", 30)
VIP_BONUS_MULTIPLIER = _get_float("VIP_BONUS_MULTIPLIER", 3.0)
VIP_FREE_PHOTOS = True
VIP_WATCH_DISCOUNT = _get_float("VIP_WATCH_DISCOUNT", 0.5)

# === GAMES ===
LOOTBOX_COST = _get_float("LOOTBOX_COST", 5.0)
LOOTBOX_REWARDS = [
    (0.30, 1),
    (0.25, 3),
    (0.20, 5),
    (0.12, 10),
    (0.08, 20),
    (0.04, 50),
```

```
    (0.01, 100),
]

DICE_MIN_BET = _get_int("DICE_MIN_BET", 1)
DICE_MAX_BET = _get_int("DICE_MAX_BET", 50)

# === QUESTS ===
DAILY_QUESTS = [
    {"type": "watch", "target": 3, "reward": 2, "desc": "\u041f\u043e\u0441\u043c\u043e\u0442\u0
440\u0438 3 \u0432\u0438\u0434\u0435\u043e"},
    {"type": "upload", "target": 1, "reward": 3, "desc": "\u0417\u0430\u0433\u0440\u0443\u0437\u0
438 1 \u043a\u043e\u043d\u0442\u0435\u043d\u0442"},
    {"type": "rate", "target": 3, "reward": 1, "desc": "\u041e\u0446\u0435\u043d\u0438 3 \u0432\
u0438\u0434\u0435\u043e"},
    {"type": "comment", "target": 2, "reward": 2, "desc": "\u041e\u0441\u0442\u0430\u0432\u044c
2 \u043a\u043e\u043c\u043c\u0435\u043d\u0442\u0430\u0440\u0438\u044f"},
    {"type": "react", "target": 5, "reward": 1, "desc": "\u041f\u043e\u0441\u0442\u0430\u0432\u0
44c 5 \u0440\u0435\u0430\u043a\u0446\u0438\u0439"},
]

PREMIUM_DAILY_QUESTS = [
    {"type": "watch", "target": 10, "reward": 8, "desc": "VIP: \u041f\u043e\u0441\u043c\u043e\u0
442\u0440\u0438 10 \u0432\u0438\u0434\u0435\u043e"},
    {"type": "comment", "target": 5, "reward": 5, "desc": "VIP: \u041e\u0441\u0442\u0430\u0432\u
044c 5 \u043a\u043e\u043c\u043c\u0435\u043d\u0442\u0430\u0440\u0438\u0432"},
]

REACTION_TYPES = ["\U0001f525", "\u2764\ufe0f", "\U0001f602", "\U0001f44d", "\U0001f4af"]

# === ANTI-SPAM COMMENTS ===
COMMENTS_PER_10_MIN = _get_int("COMMENTS_PER_10_MIN", 5)
COMMENT_MIN_INTERVAL_SEC = _get_int("COMMENT_MIN_INTERVAL_SEC", 15)

# === WEEKLY REWARDS ===
WEEKLY_TOP1_REWARD = _get_float("WEEKLY_TOP1_REWARD", 25.0)
WEEKLY_TOP2_REWARD = _get_float("WEEKLY_TOP2_REWARD", 15.0)
WEEKLY_TOP3_REWARD = _get_float("WEEKLY_TOP3_REWARD", 10.0)

# === MONETIZATION ===
PIN_OFFER_COST = _get_float("PIN_OFFER_COST", 100.0)
BUMP_VIDEO_COST = _get_float("BUMP_VIDEO_COST", 25.0)

================================================================================
FILE: db.py
================================================================================
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker, AsyncSession

from app.config import DATABASE_URL
from app.models import Base


def _fix_url(url: str) -> str:
    if url.startswith("postgres://"):
        return url.replace("postgres://", "postgresql+asyncpg://", 1)
    if url.startswith("postgresql://"):
        return url.replace("postgresql://", "postgresql+asyncpg://", 1)
    return url


engine = create_async_engine(
    _fix_url(DATABASE_URL),
    echo=False,
)

async_session = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
)
```

```
async def init_db():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)


async def reset_db():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.drop_all)
        await conn.run_sync(Base.metadata.create_all)

================================================================================
FILE: keyboards.py
================================================================================
from aiogram.types import (
    InlineKeyboardMarkup,
    InlineKeyboardButton,
    ReplyKeyboardMarkup,
    KeyboardButton,
)

from app.config import STARS_PACKAGES, REACTION_TYPES

BTN_WATCH = "\U0001f3a5 \u0421\u043c\u043e\u0442\u0440\u0435\u0442\u044c"
BTN_UPLOAD = "\U0001f4e4 \u0417\u0430\u0433\u0440\u0443\u0437\u0438\u0442\u044c"
BTN_PROFILE = "\U0001f464 \u041f\u0440\u043e\u0444\u0438\u043b\u044c"
BTN_BUY = "\U0001f4b3 \u041a\u0443\u043f\u0438\u0442\u044c \u043c\u043e\u043d\u0435\u0442\u044b"
BTN_OFFERS = "\U0001f381 \u041e\u0444\u0444\u0435\u0440\u044b"
BTN_REFERRALS = "\U0001f465 \u041f\u0440\u0438\u0433\u043b\u0430\u0441\u0438\u0442\u044c \u0434\
u0440\u0443\u0433\u0430"
BTN_BONUS = "\U0001f3c6 \u0411\u043e\u043d\u0443\u0441"
BTN_ADMIN = "\U0001f6e0 \u0410\u0434\u043c\u0438\u043d"
BTN_GAMES = "\U0001f3ae \u0418\u0433\u0440\u044b"
BTN_TOPS = "\U0001f3c6 \u0422\u043e\u043f\u044b"
BTN_QUESTS = "\U0001f3af \u041a\u0432\u0435\u0441\u0442\u044b"
BTN_VIP = "\U0001f48e VIP"
BTN_LEVEL = "\U0001f4c8 \u0423\u0440\u043e\u0432\u0435\u043d\u044c"


def main_menu(is_admin: bool = False) -> ReplyKeyboardMarkup:
    rows = [
        [KeyboardButton(text=BTN_WATCH), KeyboardButton(text=BTN_UPLOAD)],
        [KeyboardButton(text=BTN_PROFILE), KeyboardButton(text=BTN_LEVEL)],
        [KeyboardButton(text=BTN_BUY), KeyboardButton(text=BTN_VIP)],
        [KeyboardButton(text=BTN_GAMES), KeyboardButton(text=BTN_TOPS)],
        [KeyboardButton(text=BTN_QUESTS), KeyboardButton(text=BTN_OFFERS)],
        [KeyboardButton(text=BTN_REFERRALS), KeyboardButton(text=BTN_BONUS)],
    ]
    if is_admin:
        rows.append([KeyboardButton(text=BTN_ADMIN)])
    return ReplyKeyboardMarkup(keyboard=rows, resize_keyboard=True)


def rules_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [
                InlineKeyboardButton(
                    text="\u2705 \u041f\u0440\u0438\u043d\u0438\u043c\u0430\u044e",
                    callback_data="accept_rules",
                )
            ],
        ]
    )


def watch_choice_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [
```

```
                InlineKeyboardButton(
                    text="\U0001f3ac \u0412\u0438\u0434\u0435\u043e",
                    callback_data="watch_video_content",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\U0001f5bc \u0424\u043e\u0442\u043e",
                    callback_data="watch_photo_content",
                )
            ],
        ]
    )


def video_rating_keyboard(video_id: int) -> InlineKeyboardMarkup:
    stars = []
    for i in range(1, 6):
        stars.append(
            InlineKeyboardButton(
                text=f"{i}\u2b50",
                callback_data=f"rate:{video_id}:{i}",
            )
        )

    return InlineKeyboardMarkup(
        inline_keyboard=[
            stars,
            [
                InlineKeyboardButton(
                    text="\u2764\ufe0f \u0420\u0435\u0430\u043a\u0446\u0438\u0438",
                    callback_data=f"react_menu:{video_id}",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\U0001f4ac \u041a\u043e\u043c\u043c\u0435\u043d\u0442\u044b",
                    callback_data=f"comments:{video_id}",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\u270d\ufe0f \u041d\u0430\u043f\u0438\u0441\u0430\u0442\u044c \u043a\u043e\u043c\u043c\u0435\u043d\u0442",
                    callback_data=f"comment_add:{video_id}",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\u25b6\ufe0f \u0421\u043b\u0435\u0434\u0443\u044e\u0449\u0435\u0435",
                    callback_data="watch_next",
                )
            ],
        ]
    )


def photo_actions_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [
                InlineKeyboardButton(
                    text="\U0001f5bc \u0421\u043b\u0435\u0434\u0443\u044e\u0449\u0435\u0435 \u0444\u043e\u0442\u043e",
                    callback_data="watch_next_photo",
                )
            ],
        ]
    )
```

```
def reaction_menu_keyboard(video_id: int) -> InlineKeyboardMarkup:
    row = []
    for rt in REACTION_TYPES:
        row.append(
            InlineKeyboardButton(
                text=rt,
                callback_data=f"react:{video_id}:{rt}",
            )
        )

    return InlineKeyboardMarkup(
        inline_keyboard=[
            row,
            [
                InlineKeyboardButton(
                    text="\U0001f4ac \u041a\u043e\u043c\u043c\u0435\u043d\u0442\u044b",
                    callback_data=f"comments:{video_id}",
                )
            ],
        ]
    )


def buy_coins_keyboard() -> InlineKeyboardMarkup:
    buttons = []
    for key, pkg in STARS_PACKAGES.items():
        buttons.append(
            [
                InlineKeyboardButton(
                    text=f"\u2b50 {pkg['stars']} Stars \u2192 {pkg['coins']} \u043c\u043e\u043d\
u0435\u0442",
                    callback_data=f"buy:{key}",
                )
            ]
        )

    buttons.append(
        [
            InlineKeyboardButton(
                text="\U0001f4dd \u0421\u0432\u043e\u044f \u0441\u0443\u043c\u043c\u0430",
                callback_data="buy_custom",
            )
        ]
    )

    return InlineKeyboardMarkup(inline_keyboard=buttons)


def vip_buy_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [
                InlineKeyboardButton(
                    text="\U0001f48e \u041a\u0443\u043f\u0438\u0442\u044c VIP",
                    callback_data="buy_vip",
                )
            ],
        ]
    )


def games_menu_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [
                InlineKeyboardButton(
                    text="\U0001f4e6 \u041b\u0443\u0442\u0431\u043e\u043a\u0441",
                    callback_data="game_lootbox",
                )
```

```
            ],
            [
                InlineKeyboardButton(
                    text="\U0001f3b2 \u041a\u043e\u0441\u0442\u0438",
                    callback_data="game_dice",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\U0001fa99 \u041c\u043e\u043d\u0435\u0442\u043a\u0430",
                    callback_data="game_coinflip",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\U0001f522 \u0423\u0433\u0430\u0434\u0430\u0439 \u0447\u0438\u0441\u043b\u043e",
                    callback_data="game_guess",
                )
            ],
        ]
    )


def tops_menu_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [
                InlineKeyboardButton(
                    text="\U0001f3c6 \u0422\u043e\u043f \u0437\u0430\u0433\u0440\u0443\u0437\u0447\u0438\u043a\u043e\u0432",
                    callback_data="top_uploaders",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\U0001f440 \u0422\u043e\u043f \u0437\u0440\u0438\u0442\u0435\u043b\u0435\u0439",
                    callback_data="top_viewers",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\U0001f4c8 \u0422\u043e\u043f \u043f\u043e XP",
                    callback_data="top_levels",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\U0001f4b0 \u0422\u043e\u043f \u0431\u043e\u0433\u0430\u0447\u0435\u0439",
                    callback_data="top_richest",
                )
            ],
        ]
    )


def offers_list_keyboard(offers) -> InlineKeyboardMarkup:
    buttons = []
    for o in offers:
        icon = "\U0001f7e2" if o.is_active else "\U0001f534"
        buttons.append(
            [
                InlineKeyboardButton(
                    text=f"{icon} {o.title}",
                    callback_data=f"offer_open:{o.id}",
                )
            ]
        )
```

```
    return InlineKeyboardMarkup(inline_keyboard=buttons)


def offer_view_keyboard(offer_id: int, channel_url: str) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [
                InlineKeyboardButton(
                    text="\U0001f517 \u041f\u0435\u0440\u0435\u0439\u0442\u0438 \u0432 \u043a\u0
430\u043d\u0430\u043b",
                    url=channel_url,
                )
            ],
            [
                InlineKeyboardButton(
                    text="\u2705 \u041d\u0430\u0447\u0430\u0442\u044c",
                    callback_data=f"offer_start:{offer_id}",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\U0001f50d \u041f\u0440\u043e\u0432\u0435\u0440\u0438\u0442\u044c",
                    callback_data=f"offer_check:{offer_id}",
                )
            ],
        ]
    )


def quests_keyboard(quests) -> InlineKeyboardMarkup:
    buttons = []
    for q in quests:
        if q.reward_claimed:
            status = "\u2705"
        elif q.completed:
            status = "\U0001f381"
        else:
            status = "\u23f3"

        buttons.append(
            [
                InlineKeyboardButton(
                    text=f"{status} {q.quest_type}: {q.progress}/{q.target}",
                    callback_data=f"quest_claim:{q.id}" if q.completed and not q.reward_claimed
else "noop",
                )
            ]
        )
    return InlineKeyboardMarkup(inline_keyboard=buttons)


def admin_center_keyboard(is_super_admin: bool = False) -> InlineKeyboardMarkup:
    buttons = [
        [InlineKeyboardButton(text="\U0001f4ca \u041e\u0447\u0435\u0440\u0435\u0434\u044c", call
back_data="admin_queue_info")],
        [InlineKeyboardButton(text="\U0001f4ca \u0421\u0442\u0430\u0442\u0438\u0441\u0442\u0438\
u043a\u0430+", callback_data="admin_extended_stats")],
        [InlineKeyboardButton(text="\U0001f4dc \u041b\u043e\u0433\u0438", callback_data="admin_l
ogs")],
        [InlineKeyboardButton(text="\U0001f4cb \u041c\u043e\u0434\u0435\u0440\u0430\u0446\u0438\
u044f", callback_data="admin_get_pending")],
        [InlineKeyboardButton(text="\u2705 \u041e\u0434\u043e\u0431\u0440\u0438\u0442\u044c \u04
32\u0441\u0451", callback_data="admin_approve_all")],
        [InlineKeyboardButton(text="\U0001f381 \u041e\u0444\u0444\u0435\u0440\u044b", callback_d
ata="admin_offers_menu")],
    ]
    if is_super_admin:
        buttons.append(
            [
                InlineKeyboardButton(
```

```
                    text="\U0001f451 \u0423\u043f\u0440\u0430\u0432\u043b\u0435\u043d\u0438\u043
5 \u0430\u0434\u043c\u0438\u043d\u0430\u043c\u0438",
                    callback_data="admin_manage_admins",
                )
            ]
        )
    return InlineKeyboardMarkup(inline_keyboard=buttons)


def moderation_keyboard(video_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [
                InlineKeyboardButton(
                    text="\u2705 \u041e\u0434\u043e\u0431\u0440\u0438\u0442\u044c",
                    callback_data=f"mod_approve:{video_id}",
                ),
                InlineKeyboardButton(
                    text="\u274c \u041e\u0442\u043a\u043b\u043e\u043d\u0438\u0442\u044c",
                    callback_data=f"mod_reject:{video_id}",
                ),
            ],
            [
                InlineKeyboardButton(
                    text="\u23ed \u041f\u0440\u043e\u043f\u0443\u0441\u0442\u0438\u0442\u044c",
                    callback_data="admin_get_pending",
                )
            ],
        ]
    )


def rejection_reason_keyboard(video_id: int) -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [
                InlineKeyboardButton(
                    text="\U0001f503 \u0414\u0443\u0431\u043b\u0438\u043a\u0430\u0442",
                    callback_data=f"reject_reason:{video_id}:duplicate",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\U0001f6ab \u041d\u0435 \u043f\u043e \u0442\u0435\u043c\u0430\u0442\u0
438\u043a\u0435",
                    callback_data=f"reject_reason:{video_id}:off_topic",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\u2753 \u0414\u0440\u0443\u0433\u043e\u0435",
                    callback_data=f"reject_reason:{video_id}:other",
                )
            ],
        ]
    )


def admin_after_action_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [
                InlineKeyboardButton(
                    text="\u25b6\ufe0f \u0421\u043b\u0435\u0434\u0443\u044e\u0449\u0438\u0439",
                    callback_data="admin_get_pending",
                )
            ],
            [
                InlineKeyboardButton(
                    text="\U0001f519 \u0410\u0434\u043c\u0438\u043d-\u0446\u0435\u043d\u0442\u04
```

```
40",
                    callback_data="admin_center",
                )
            ],
        ]
    )


def admin_offers_menu_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [InlineKeyboardButton(text="\u2795 \u0421\u043e\u0437\u0434\u0430\u0442\u044c", call
back_data="admin_offer_create")],
            [InlineKeyboardButton(text="\U0001f4cb \u0421\u043f\u0438\u0441\u043e\u043a", callba
ck_data="admin_offer_list")],
            [InlineKeyboardButton(text="\U0001f519 \u041d\u0430\u0437\u0430\u0434", callback_dat
a="admin_center")],
        ]
    )


def admin_offer_list_keyboard(offers) -> InlineKeyboardMarkup:
    buttons = []
    for o in offers:
        icon = "\U0001f7e2" if o.is_active else "\U0001f534"
        buttons.append(
            [
                InlineKeyboardButton(
                    text=f"{icon} {o.title}",
                    callback_data=f"admin_offer_toggle:{o.id}",
                )
            ]
        )
    buttons.append(
        [
            InlineKeyboardButton(
                text="\U0001f519 \u041d\u0430\u0437\u0430\u0434",
                callback_data="admin_offers_menu",
            )
        ]
    )
    return InlineKeyboardMarkup(inline_keyboard=buttons)


def admin_manage_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [InlineKeyboardButton(text="\U0001f4cb \u0421\u043f\u0438\u0441\u043e\u043a", callba
ck_data="admm_list")],
            [InlineKeyboardButton(text="\u2795 \u0414\u043e\u0431\u0430\u0432\u0438\u0442\u044c"
, callback_data="admm_add")],
            [InlineKeyboardButton(text="\u2796 \u0423\u0434\u0430\u043b\u0438\u0442\u044c", call
back_data="admm_remove")],
            [InlineKeyboardButton(text="\U0001f519 \u041d\u0430\u0437\u0430\u0434", callback_dat
a="admin_center")],
        ]
    )


def back_to_admin_manage_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        inline_keyboard=[
            [InlineKeyboardButton(text="\U0001f519 \u041d\u0430\u0437\u0430\u0434", callback_dat
a="admin_manage_admins")]
        ]
    )

================================================================================
FILE: logger.py
================================================================================
```

```
import json
import logging
import traceback
from datetime import datetime


def get_logger(name: str) -> logging.Logger:
    return logging.getLogger(name)


def _make_payload(message: str, **kwargs) -> str:
    payload = {
        "time": datetime.utcnow().isoformat() + "Z",
        "message": message,
    }
    payload.update(kwargs)
    return json.dumps(payload, ensure_ascii=False)


def setup_logging(level: int = logging.INFO):
    root = logging.getLogger()
    root.setLevel(level)

    while root.handlers:
        root.handlers.pop()

    handler = logging.StreamHandler()
    handler.setLevel(level)
    handler.setFormatter(logging.Formatter("%(message)s"))
    root.addHandler(handler)


def log_info(logger: logging.Logger, message: str, **kwargs):
    logger.info(_make_payload(message, **kwargs))


def log_warning(logger: logging.Logger, message: str, **kwargs):
    logger.warning(_make_payload(message, **kwargs))


def log_error(logger: logging.Logger, message: str, **kwargs):
    logger.error(_make_payload(message, **kwargs))


def log_exception(logger: logging.Logger, message: str, **kwargs):
    kwargs["traceback"] = traceback.format_exc()
    logger.error(_make_payload(message, **kwargs))


def log_event(logger: logging.Logger, level: int, message: str, **kwargs):
    logger.log(level, _make_payload(message, **kwargs))

================================================================================
FILE: main.py
================================================================================
import asyncio
import logging
from datetime import datetime

from aiohttp import web
from aiogram import Bot, Dispatcher
from aiogram.client.default import DefaultBotProperties
from aiogram.enums import ParseMode

from app.config import (
    BOT_TOKEN,
    OFFER_BROADCAST_INTERVAL_HOURS,
    LOG_CHAT_ID,
    PORT,
)
```

```
from app.db import engine, Base, async_session
from app.user_handlers import router as user_router
from app.admin_handlers import router as admin_router
from app.services import (
    get_active_offers,
    get_users_without_offer,
    reward_weekly_top_users,
)
from app.keyboards import offer_view_keyboard
from app.logger import setup_logging, get_logger, log_info, log_warning, log_exception

setup_logging()
logger = get_logger(__name__)


async def tg_log(bot: Bot, text: str):
    if not LOG_CHAT_ID:
        return

    chat_ids = [s.strip() for s in str(LOG_CHAT_ID).split(",") if s.strip()]
    for cid in chat_ids:
        try:
            await bot.send_message(int(cid), text, parse_mode="HTML")
        except Exception as e:
            log_warning(logger, "[illegible] Telegram", chat_id=cid, error=
str(e))


async def offer_broadcaster(bot: Bot):
    interval = max(60, int(OFFER_BROADCAST_INTERVAL_HOURS * 3600))
    log_info(logger, "[illegible]", interval_seconds=interval)

    while True:
        await asyncio.sleep(interval)

        try:
            async with async_session() as session:
                offers = await get_active_offers(session)
                sent_count = 0

                for offer in offers:
                    users = await get_users_without_offer(session, offer.id)

                    for user in users:
                        try:
                            text = (
                                f"\U0001f381 <b>{offer.title}</b>\n\n"
                                f"{offer.description}\n\n"
                                f"\U0001f4b0 \u0412\u0441\u0435\u0433\u043e \u043c\u043e\u0436\u0
043d\u043e \u043f\u043e\u043b\u0443\u0447\u0438\u0442\u044c: <b>40</b> \u043c\u043e\u043d\u0435\
u0442"
                            )
                            await bot.send_message(
                                user.telegram_id,
                                text,
                                parse_mode="HTML",
                                reply_markup=offer_view_keyboard(offer.id, offer.channel_url),
                            )
                            sent_count += 1
                        except Exception:
                            pass

                        await asyncio.sleep(0.1)

                log_info(
                    logger,
                    "[illegible]",
                    offers_count=len(offers),
                    sent_count=sent_count,
                )
```

```
        except asyncio.CancelledError:
            log_info(logger, "[redacted]")
            raise
        except Exception:
            log_exception(logger, "[redacted]")
            try:
                await tg_log(bot, "\u26a0\ufe0f <b>OFFER BROADCAST ERROR</b>")
            except Exception:
                pass


async def weekly_rewards_worker(bot: Bot):
    log_info(logger, "[redacted]")

    while True:
        await asyncio.sleep(3600)

        try:
            dt = datetime.utcnow()
            if dt.weekday() == 0 and dt.hour == 0:
                async with async_session() as session:
                    rewarded = await reward_weekly_top_users(session)

                    if rewarded:
                        lines = ["\U0001f3c6 <b>Weekly rewards paid</b>\n"]

                        for idx, (user, reward) in enumerate(rewarded, start=1):
                            lines.append(f"{idx}. <code>{user.telegram_id}</code> +{reward}")

                            try:
                                await bot.send_message(
                                    user.telegram_id,
                                    f"\U0001f3c6 \u0412\u044b \u0432\u043e\u0448\u043b\u0438 \u0
432 weekly top-{idx}!\n"
                                    f"\U0001f4b0 \u041d\u0430\u0433\u0440\u0430\u0434\u0430: <b>
{reward}</b>",
                                    parse_mode="HTML",
                                )
                            except Exception:
                                pass

                        await tg_log(bot, "\n".join(lines))
                        log_info(
                            logger,
                            "Weekly rewards [redacted]",
                            rewarded_count=len(rewarded),
                        )

                await asyncio.sleep(3700)

        except asyncio.CancelledError:
            log_info(logger, "[redacted]")
            raise
        except Exception:
            log_exception(logger, "[redacted]")
            try:
                await tg_log(bot, "\u26a0\ufe0f <b>WEEKLY REWARD ERROR</b>")
            except Exception:
                pass


async def on_startup(app):
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)

    log_info(logger, "[redacted]")


async def handle_health(request):
```

```
    return web.Response(text="OK")


async def handle_health_head(request):
    return web.Response(
        status=200,
        content_type="text/plain",
        headers={"Content-Length": "2"},
    )


async def main():
    if not BOT_TOKEN:
        raise RuntimeError("BOT_TOKEN is empty")

    bot = Bot(
        token=BOT_TOKEN,
        default=DefaultBotProperties(parse_mode=ParseMode.HTML),
    )

    await bot.delete_webhook(drop_pending_updates=True)
    log_info(logger, "Webhook ███████, ████ ██████████ ■ polling")

    try:
        await tg_log(bot, "\U0001f7e2 <b>Bot started</b>")
    except Exception:
        pass

    dp = Dispatcher()
    dp.include_router(user_router)
    dp.include_router(admin_router)

    app = web.Application()
    app.router.add_get("/", handle_health)
    app.router.add_get("/health", handle_health)
    app.router.add_head("/", handle_health_head)
    app.router.add_head("/health", handle_health_head)
    app.on_startup.append(on_startup)

    runner = web.AppRunner(app)
    await runner.setup()

    port = int(PORT or 10000)
    site = web.TCPSite(runner, "0.0.0.0", port)
    await site.start()

    log_info(logger, "HTTP ███████ ████████", port=port)

    offer_task = asyncio.create_task(offer_broadcaster(bot))
    weekly_task = asyncio.create_task(weekly_rewards_worker(bot))

    try:
        log_info(logger, "Polling ████████")
        await dp.start_polling(bot)
    finally:
        log_info(logger, "██████████ ███████████")

        for task in (offer_task, weekly_task):
            task.cancel()

        await asyncio.gather(offer_task, weekly_task, return_exceptions=True)

        await runner.cleanup()
        await bot.session.close()
        await engine.dispose()

        log_info(logger, "███████████ ███████ ██████████")


if __name__ == "__main__":
```

```
    asyncio.run(main())

================================================================================
FILE: make_project_pdf.py
================================================================================
import os
from pathlib import Path
from datetime import datetime

from reportlab.lib.pagesizes import A4
from reportlab.pdfbase.pdfmetrics import stringWidth
from reportlab.pdfgen import canvas


PROJECT_ROOT = Path(".")
OUTPUT_FILE = "project_dump.pdf"

MAX_PAGES = 90
MAX_SIZE_MB = 15

INCLUDE_EXTENSIONS = {
    ".py",
    ".txt",
    ".md",
    ".json",
    ".yaml",
    ".yml",
    ".toml",
    ".ini",
    ".env",
    ".sql",
}

SPECIAL_FILENAMES = {
    "Dockerfile",
    "Procfile",
    "requirements.txt",
    "runtime.txt",
    ".env",
}

EXCLUDED_DIRS = {
    ".git",
    ".venv",
    "venv",
    "__pycache__",
    "node_modules",
    ".idea",
    ".vscode",
    "dist",
    "build",
    ".pytest_cache",
    ".mypy_cache",
    ".ruff_cache",
    ".cache",
}

EXCLUDED_FILES = {
    OUTPUT_FILE,
}

MAX_FILE_SIZE_KB = 300


def is_text_candidate(path: Path) -> bool:
    if path.name in SPECIAL_FILENAMES:
        return True
    if path.suffix.lower() in INCLUDE_EXTENSIONS:
        return True
    if path.name.startswith(".env"):
```

```
        return True
    return False


def should_skip(path: Path) -> bool:
    if path.is_dir():
        return True

    if path.name in EXCLUDED_FILES:
        return True

    parts = set(path.parts)
    if parts & EXCLUDED_DIRS:
        return True

    if not is_text_candidate(path):
        return True

    try:
        if path.stat().st_size > MAX_FILE_SIZE_KB * 1024:
            return True
    except Exception:
        return True

    return False


def iter_project_files(root: Path):
    files = []
    for path in root.rglob("*"):
        if should_skip(path):
            continue
        files.append(path)
    files.sort(key=lambda p: str(p).lower())
    return files


def safe_read_text(path: Path) -> str:
    for enc in ("utf-8", "utf-8-sig", "cp1251", "latin-1"):
        try:
            return path.read_text(encoding=enc)
        except Exception:
            pass
    return "[[FAILED TO READ FILE]]"


def wrap_line(line: str, font_name: str, font_size: int, max_width: float):
    if not line:
        return [""]

    if stringWidth(line, font_name, font_size) <= max_width:
        return [line]

    parts = []
    current = ""

    for ch in line:
        test = current + ch
        if stringWidth(test, font_name, font_size) <= max_width:
            current = test
        else:
            if current:
                parts.append(current)
            current = ch

    if current:
        parts.append(current)

    return parts
```

```
def build_blocks(root: Path):
    files = iter_project_files(root)
    blocks = []

    header = [
        "<PROJECT PDF DUMP>",
        f"Generated at: {datetime.utcnow().isoformat()} UTC",
        f"Root: {root.resolve()}",
        f"Files included: {len(files)}",
        "",
    ]
    blocks.append("\n".join(header))

    for path in files:
        rel = path.relative_to(root)
        content = safe_read_text(path).rstrip()

        block = [
            "=" * 80,
            f"FILE: {rel}",
            "=" * 80,
            content,
            "",
        ]
        blocks.append("\n".join(block))

    return blocks, files


def render_pdf(blocks, output_path: str):
    page_width, page_height = A4

    configs = [
        {"font_size": 9, "line_gap": 11, "margin": 36},
        {"font_size": 8, "line_gap": 9, "margin": 26},
        {"font_size": 7, "line_gap": 8, "margin": 18},
        {"font_size": 6, "line_gap": 7, "margin": 14},
    ]

    best_result = None

    for cfg in configs:
        c = canvas.Canvas(output_path, pagesize=A4)
        font_name = "Courier"
        font_size = cfg["font_size"]
        line_gap = cfg["line_gap"]
        margin = cfg["margin"]

        usable_width = page_width - margin * 2
        y = page_height - margin
        pages = 1

        c.setFont(font_name, font_size)

        def new_page():
            nonlocal y, pages
            c.showPage()
            c.setFont(font_name, font_size)
            y = page_height - margin
            pages += 1

        for block in blocks:
            for raw_line in block.splitlines():
                wrapped = wrap_line(raw_line, font_name, font_size, usable_width)
                for line in wrapped:
                    if y < margin:
                        new_page()
                    c.drawString(margin, y, line)
                    y -= line_gap
```

```
            if y < margin:
                new_page()
            y -= line_gap

        c.save()

        size_bytes = os.path.getsize(output_path)
        size_mb = size_bytes / (1024 * 1024)

        best_result = {
            "pages": pages,
            "size_bytes": size_bytes,
            "size_mb": size_mb,
            "config": cfg,
        }

        if pages <= MAX_PAGES and size_mb <= MAX_SIZE_MB:
            return best_result

    return best_result


def main():
    blocks, files = build_blocks(PROJECT_ROOT)
    result = render_pdf(blocks, OUTPUT_FILE)

    print("=" * 60)
    print(f"PDF created: {OUTPUT_FILE}")
    print(f"Files included: {len(files)}")
    print(f"Pages: {result['pages']}")
    print(f"Size: {result['size_mb']:.2f} MB")
    print(f"Config used: {result['config']}")
    print("=" * 60)

    if result["pages"] > MAX_PAGES:
        print(f"WARNING: PDF exceeds page limit ({result['pages']} > {MAX_PAGES})")

    if result["size_mb"] > MAX_SIZE_MB:
        print(f"WARNING: PDF exceeds size limit ({result['size_mb']:.2f} MB > {MAX_SIZE_MB} MB)"
)


if __name__ == "__main__":
    main()

================================================================================
FILE: migrate.py
================================================================================
import asyncio
from sqlalchemy import text
from app.db import engine


MIGRATIONS = [
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS created_by_user_id INTEGER NULL;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS offer_kind VARCHAR(30) DEFAULT 'system';
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS promotion_tier VARCHAR(30) DEFAULT 'basic';
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS payment_method VARCHAR(20) NULL;
```

```
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS promo_price_coins NUMERIC(12,2) DEFAULT 0;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS promo_price_rub INTEGER DEFAULT 0;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS promo_price_stars INTEGER DEFAULT 0;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS priority_level INTEGER DEFAULT 10;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS is_featured BOOLEAN DEFAULT FALSE;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS moderation_comment TEXT NULL;
    """,
    """
    ALTER TABLE offers
    ADD COLUMN IF NOT EXISTS approved_at TIMESTAMPTZ NULL;
    """,
    """
    CREATE INDEX IF NOT EXISTS ix_offers_created_by_user_id ON offers (created_by_user_id);
    """,
    """
    CREATE INDEX IF NOT EXISTS ix_offers_status ON offers (status);
    """,
]


async def main():
    async with engine.begin() as conn:
        for sql in MIGRATIONS:
            await conn.execute(text(sql))
    await engine.dispose()
    print("Migrations applied successfully")


if __name__ == "__main__":
    asyncio.run(main())

================================================================================
FILE: models.py
================================================================================
from datetime import datetime
from decimal import Decimal

from sqlalchemy import (
    BigInteger,
    String,
    Numeric,
    Boolean,
    Integer,
    DateTime,
    ForeignKey,
    UniqueConstraint,
    Text,
    Date,
)
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship
```

```
class Base(DeclarativeBase):
    pass


class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    telegram_id: Mapped[int] = mapped_column(
        BigInteger,
        unique=True,
        nullable=False,
        index=True,
    )
    username: Mapped[str | None] = mapped_column(String(255), nullable=True, index=True)
    first_name: Mapped[str | None] = mapped_column(String(255), nullable=True)
    last_name: Mapped[str | None] = mapped_column(String(255), nullable=True)

    balance: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=0)
    status: Mapped[str] = mapped_column(String(50), default="active")
    is_admin: Mapped[bool] = mapped_column(Boolean, default=False)
    agreed_to_rules: Mapped[bool] = mapped_column(Boolean, default=False)

    referral_code: Mapped[str | None] = mapped_column(
        String(50),
        unique=True,
        nullable=True,
    )
    referred_by_user_id: Mapped[int | None] = mapped_column(
        ForeignKey("users.id"),
        nullable=True,
    )
    referral_earnings: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=0)

    last_bonus_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)

    xp: Mapped[int] = mapped_column(Integer, default=0)
    level: Mapped[int] = mapped_column(Integer, default=1)

    vip_until: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)

    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    videos: Mapped[list["Video"]] = relationship(back_populates="uploader")
    views: Mapped[list["VideoView"]] = relationship(back_populates="user")
    ratings: Mapped[list["VideoRating"]] = relationship(back_populates="user")
    payments: Mapped[list["Payment"]] = relationship(back_populates="user")
    comments: Mapped[list["Comment"]] = relationship(back_populates="user")
    reactions: Mapped[list["ContentReaction"]] = relationship(back_populates="user")
    game_history: Mapped[list["GameHistory"]] = relationship(back_populates="user")
    quest_progress: Mapped[list["DailyQuestProgress"]] = relationship(back_populates="user")


class Video(Base):
    __tablename__ = "videos"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    uploader_user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)

    content_type: Mapped[str] = mapped_column(String(20), default="video")
    telegram_file_id: Mapped[str] = mapped_column(Text, nullable=False)
    telegram_file_unique_id: Mapped[str] = mapped_column(
        String(255),
        nullable=False,
        index=True,
    )

    status: Mapped[str] = mapped_column(String(20), default="pending")
    duration_seconds: Mapped[int | None] = mapped_column(Integer, nullable=True)
    file_size: Mapped[int | None] = mapped_column(BigInteger, nullable=True)
```

```
    rejection_reason: Mapped[str | None] = mapped_column(String(255), nullable=True)

    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    uploader: Mapped["User"] = relationship(back_populates="videos")
    views: Mapped[list["VideoView"]] = relationship(back_populates="video")
    ratings: Mapped[list["VideoRating"]] = relationship(back_populates="video")
    comments: Mapped[list["Comment"]] = relationship(back_populates="video")
    reactions: Mapped[list["ContentReaction"]] = relationship(back_populates="video")


class VideoView(Base):
    __tablename__ = "video_views"
    __table_args__ = (
        UniqueConstraint("user_id", "video_id", name="uq_user_video_view"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    video_id: Mapped[int] = mapped_column(ForeignKey("videos.id"), nullable=False)
    watched_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="views")
    video: Mapped["Video"] = relationship(back_populates="views")


class VideoRating(Base):
    __tablename__ = "video_ratings"
    __table_args__ = (
        UniqueConstraint("user_id", "video_id", name="uq_user_video_rating"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    video_id: Mapped[int] = mapped_column(ForeignKey("videos.id"), nullable=False)
    rating: Mapped[int] = mapped_column(Integer, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="ratings")
    video: Mapped["Video"] = relationship(back_populates="ratings")


class Payment(Base):
    __tablename__ = "payments"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)

    payload: Mapped[str] = mapped_column(String(255), unique=True, nullable=False)
    stars_amount: Mapped[int] = mapped_column(Integer, nullable=False)
    coins_amount: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
    status: Mapped[str] = mapped_column(String(50), default="pending")

    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="payments")


class Offer(Base):
    __tablename__ = "offers"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False)
    channel_url: Mapped[str] = mapped_column(Text, nullable=False)

    reward_preview: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=5)
    reward_final: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=35)
    penalty_unsubscribe: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=40)
```

```
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)


class OfferParticipation(Base):
    __tablename__ = "offer_participations"
    __table_args__ = (
        UniqueConstraint("user_id", "offer_id", name="uq_user_offer"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    offer_id: Mapped[int] = mapped_column(ForeignKey("offers.id"), nullable=False)

    status: Mapped[str] = mapped_column(String(50), default="started")
    reward_given: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=0)

    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    checked_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)


class Comment(Base):
    __tablename__ = "comments"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    video_id: Mapped[int] = mapped_column(ForeignKey("videos.id"), nullable=False)

    text: Mapped[str] = mapped_column(Text, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="comments")
    video: Mapped["Video"] = relationship(back_populates="comments")


class ContentReaction(Base):
    __tablename__ = "content_reactions"
    __table_args__ = (
        UniqueConstraint("user_id", "video_id", name="uq_user_video_reaction"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    video_id: Mapped[int] = mapped_column(ForeignKey("videos.id"), nullable=False)

    reaction_type: Mapped[str] = mapped_column(String(10), nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="reactions")
    video: Mapped["Video"] = relationship(back_populates="reactions")


class GameHistory(Base):
    __tablename__ = "game_history"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)

    game_type: Mapped[str] = mapped_column(String(50), nullable=False)
    bet: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=0)
    result: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=0)
    details: Mapped[str | None] = mapped_column(Text, nullable=True)

    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="game_history")


class DailyQuestProgress(Base):
    __tablename__ = "daily_quest_progress"
```

```
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)

    quest_type: Mapped[str] = mapped_column(String(50), nullable=False)
    quest_date: Mapped[datetime] = mapped_column(Date, nullable=False)

    progress: Mapped[int] = mapped_column(Integer, default=0)
    target: Mapped[int] = mapped_column(Integer, nullable=False)
    reward: Mapped[Decimal] = mapped_column(Numeric(10, 2), default=0)

    completed: Mapped[bool] = mapped_column(Boolean, default=False)
    reward_claimed: Mapped[bool] = mapped_column(Boolean, default=False)

    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    user: Mapped["User"] = relationship(back_populates="quest_progress")

================================================================================
FILE: services.py
================================================================================
import uuid
import random
from datetime import datetime, timedelta
from decimal import Decimal, ROUND_DOWN

from sqlalchemy import select, func
from sqlalchemy.ext.asyncio import AsyncSession

from app.models import (
    User,
    Video,
    VideoView,
    VideoRating,
    Payment,
    Offer,
    OfferParticipation,
    Comment,
    ContentReaction,
    GameHistory,
    DailyQuestProgress,
)
from app.config import (
    STARTING_BALANCE,
    WATCH_COST,
    UPLOAD_REWARD,
    REFERRAL_REWARD_INVITER,
    REFERRAL_REWARD_NEW_USER,
    STARS_PACKAGES,
    STARS_TO_COINS_RATE,
    MONEY_PACKAGES,
    LEVEL_XP_BASE,
    LEVEL_XP_MULTIPLIER,
    XP_PER_WATCH,
    XP_PER_UPLOAD,
    XP_PER_RATING,
    XP_PER_COMMENT,
    XP_PER_REACTION,
    XP_PER_GAME,
    VIP_PRICE_STARS,
    VIP_DURATION_DAYS,
    VIP_BONUS_MULTIPLIER,
    VIP_WATCH_DISCOUNT,
    LOOTBOX_COST,
    LOOTBOX_REWARDS,
    DICE_MIN_BET,
    DICE_MAX_BET,
    DAILY_QUESTS,
    PREMIUM_DAILY_QUESTS,
    REACTION_TYPES,
```

```
    COMMENTS_PER_10_MIN,
    COMMENT_MIN_INTERVAL_SEC,
    WEEKLY_TOP1_REWARD,
    WEEKLY_TOP2_REWARD,
    WEEKLY_TOP3_REWARD,
    PIN_OFFER_COST,
    BUMP_VIDEO_COST,
)

BONUS_AMOUNT = Decimal("1.00")
BONUS_COOLDOWN_HOURS = 4
PHOTO_UPLOAD_REWARD = Decimal("0.10")
FREE_PHOTO_LIMIT_PER_4H = 20
OFFER_STEP_1_REWARD = Decimal("5.00")
OFFER_PENALTY = Decimal("40.00")


def to_decimal(value) -> Decimal:
    if isinstance(value, Decimal):
        return value
    return Decimal(str(value))


def round_coin(v) -> Decimal:
    return to_decimal(v).quantize(Decimal("0.01"), rounding=ROUND_DOWN)


# ========== FORMAT ==========

def format_duration(seconds):
    if not seconds:
        return "0 \u0441\u0435\u043a"

    h = seconds // 3600
    m = (seconds % 3600) // 60
    s = seconds % 60

    if h > 0:
        return f"{h}\u0447 {m:02d}\u043c {s:02d}\u0441"
    if m > 0:
        return f"{m}\u043c {s:02d}\u0441"
    return f"{s}\u0441"


def format_file_size(b):
    if not b:
        return "0"
    if b >= 1048576:
        return f"{b / 1048576:.1f}MB"
    if b >= 1024:
        return f"{b / 1024:.1f}KB"
    return f"{b}B"


# ========== LEVELS ==========

def calc_level_info(total_xp):
    level = 1
    xp_rem = total_xp

    while True:
        needed = int(LEVEL_XP_BASE * (LEVEL_XP_MULTIPLIER ** (level - 1)))
        if xp_rem < needed:
            return level, xp_rem, needed
        xp_rem -= needed
        level += 1


async def add_xp(session, user, amount):
    user.xp += int(amount)
```

```
    new_level, _, _ = calc_level_info(user.xp)
    old_level = user.level
    user.level = new_level
    await session.commit()
    return new_level > old_level, new_level


# ========== VIP ==========

def is_vip(user):
    if not user.vip_until:
        return False
    return user.vip_until > datetime.utcnow()


def get_watch_cost_for_user(user):
    base = to_decimal(WATCH_COST)
    if is_vip(user):
        discounted = base * to_decimal(VIP_WATCH_DISCOUNT)
        return round_coin(discounted)
    return round_coin(base)


async def activate_vip(session, user):
    now = datetime.utcnow()
    if user.vip_until and user.vip_until > now:
        user.vip_until = user.vip_until + timedelta(days=VIP_DURATION_DAYS)
    else:
        user.vip_until = now + timedelta(days=VIP_DURATION_DAYS)

    await session.commit()
    await session.refresh(user)
    return user


async def create_vip_payment(session, user):
    payload = f"vip:{user.telegram_id}:{uuid.uuid4().hex[:12]}"
    payment = Payment(
        user_id=user.id,
        payload=payload,
        stars_amount=VIP_PRICE_STARS,
        coins_amount=Decimal("0"),
        status="pending",
    )
    session.add(payment)
    await session.commit()
    await session.refresh(payment)
    return payment


# ========== ADMINS ==========

async def get_user_by_username(session, username):
    clean = username.strip().lstrip("@").lower()
    stmt = select(User).where(func.lower(User.username) == clean)
    return (await session.execute(stmt)).scalar_one_or_none()


async def set_user_admin(session, user, value):
    user.is_admin = bool(value)
    await session.commit()
    await session.refresh(user)
    return user


async def get_db_admins(session):
    stmt = select(User).where(User.is_admin == True).order_by(User.id)
    return list((await session.execute(stmt)).scalars().all())
```

```
# ========== USER ==========

async def get_user(session, telegram_id):
    stmt = select(User).where(User.telegram_id == telegram_id)
    return (await session.execute(stmt)).scalar_one_or_none()


async def get_user_by_id(session, user_id):
    return (await session.execute(select(User).where(User.id == user_id))).scalar_one_or_none()


async def get_user_by_referral_code(session, code):
    return (await session.execute(select(User).where(User.referral_code == code))).scalar_one_or
_none()


async def get_all_users(session):
    return list(
        (await session.execute(select(User).where(User.agreed_to_rules == True))).scalars().all(
)
    )


async def get_or_create_user(
    session,
    telegram_id,
    username=None,
    first_name=None,
    last_name=None,
    referral_code=None,
):
    user = await get_user(session, telegram_id)

    if user:
        user.username = username
        user.first_name = first_name
        user.last_name = last_name
        await session.commit()
        await session.refresh(user)
        return user, False

    referred_by = None
    inviter = None

    if referral_code:
        inviter = await get_user_by_referral_code(session, referral_code)
        if inviter and inviter.telegram_id != telegram_id:
            referred_by = inviter.id

    user = User(
        telegram_id=telegram_id,
        username=username,
        first_name=first_name,
        last_name=last_name,
        balance=to_decimal(STARTING_BALANCE),
        referral_code=uuid.uuid4().hex[:8],
        referred_by_user_id=referred_by,
        referral_earnings=Decimal("0"),
        is_admin=False,
        xp=0,
        level=1,
    )
    session.add(user)
    await session.flush()

    if inviter:
        inviter.balance += to_decimal(REFERRAL_REWARD_INVITER)
        inviter.referral_earnings += to_decimal(REFERRAL_REWARD_INVITER)
        user.balance += to_decimal(REFERRAL_REWARD_NEW_USER)
```

```
    await session.commit()
    await session.refresh(user)
    return user, True


async def agree_to_rules(session, telegram_id):
    user = await get_user(session, telegram_id)
    if user:
        user.agreed_to_rules = True
        await session.commit()
        await session.refresh(user)
    return user


async def claim_daily_bonus(session, telegram_id):
    user = await get_user(session, telegram_id)
    if not user:
        return False, "\u041d\u0435 \u043d\u0430\u0439\u0434\u0435\u043d."

    now = datetime.utcnow()

    if user.last_bonus_at:
        nxt = user.last_bonus_at + timedelta(hours=BONUS_COOLDOWN_HOURS)
        if now < nxt:
            rem = nxt - now
            h = int(rem.total_seconds()) // 3600
            m = (int(rem.total_seconds()) % 3600) // 60
            return False, f"\u0427\u0435\u0440\u0435\u0437 {h}\u0447 {m}\u043c."

    bonus = BONUS_AMOUNT
    if is_vip(user):
        bonus = round_coin(bonus * to_decimal(VIP_BONUS_MULTIPLIER))

    user.balance += bonus
    user.last_bonus_at = now

    await session.commit()
    await session.refresh(user)

    vip_mark = " (VIP bonus)" if is_vip(user) else ""
    return True, f"+{bonus}{vip_mark}. \u0411\u0430\u043b\u0430\u043d\u0441: {user.balance}"


async def count_referrals(session, user_id):
    return (
        await session.execute(
            select(func.count(User.id)).where(User.referred_by_user_id == user_id)
        )
    ).scalar_one()


# ========== PAYMENTS ==========

async def create_payment(session, user, package_key):
    pkg = STARS_PACKAGES.get(package_key)
    if not pkg:
        return None

    payload = f"{package_key}:{user.telegram_id}:{uuid.uuid4().hex[:12]}"
    p = Payment(
        user_id=user.id,
        payload=payload,
        stars_amount=pkg["stars"],
        coins_amount=to_decimal(pkg["coins"]),
        status="pending",
    )
    session.add(p)
    await session.commit()
    await session.refresh(p)
    return p
```

```
async def create_custom_payment(session, user, stars):
    coins = round_coin(to_decimal(stars) * to_decimal(STARS_TO_COINS_RATE))
    payload = f"custom:{user.telegram_id}:{uuid.uuid4().hex[:12]}"
    p = Payment(
        user_id=user.id,
        payload=payload,
        stars_amount=stars,
        coins_amount=coins,
        status="pending",
    )
    session.add(p)
    await session.commit()
    await session.refresh(p)
    return p


async def create_money_payment(session, user, package_key):
    pkg = MONEY_PACKAGES.get(package_key)
    if not pkg:
        return None

    payload = f"money:{package_key}:{user.telegram_id}:{uuid.uuid4().hex[:12]}"
    p = Payment(
        user_id=user.id,
        payload=payload,
        stars_amount=0,
        coins_amount=to_decimal(pkg["coins"]),
        status="pending",
    )
    session.add(p)
    await session.commit()
    await session.refresh(p)
    return p


async def get_payment_by_payload(session, payload):
    return (
        await session.execute(select(Payment).where(Payment.payload == payload))
    ).scalar_one_or_none()


async def apply_successful_payment(session, payload):
    payment = await get_payment_by_payload(session, payload)
    if not payment:
        return False, "\u041d\u0435 \u043d\u0430\u0439\u0434\u0435\u043d."
    if payment.status == "paid":
        return True, "\u0423\u0436\u0435 \u043e\u0431\u0440\u0430\u0431\u043e\u0442\u0430\u043d.
"

    user = await get_user_by_id(session, payment.user_id)
    if not user:
        return False, "\u041f\u043e\u043b\u044c\u0437\u043e\u0432\u0430\u0442\u0435\u043b\u044c
\u043d\u0435 \u043d\u0430\u0439\u0434\u0435\u043d."

    payment.status = "paid"

    if payload.startswith("vip:"):
        await activate_vip(session, user)
        return True, f"VIP \u0434\u043e {user.vip_until.strftime('%d.%m.%Y')}!"

    user.balance += payment.coins_amount
    await session.commit()
    await session.refresh(user)
    return True, f"+{payment.coins_amount}. \u0411\u0430\u043b\u0430\u043d\u0441: {user.balance}
"


# ========== OFFERS ==========
```

```
async def create_offer(session, title, description, channel_url):
    o = Offer(
        title=title,
        description=description,
        channel_url=channel_url,
        reward_preview=OFFER_STEP_1_REWARD,
        reward_final=Decimal("35"),
        penalty_unsubscribe=OFFER_PENALTY,
        is_active=True,
    )
    session.add(o)
    await session.commit()
    await session.refresh(o)
    return o


async def get_active_offers(session):
    return list(
        (
            await session.execute(
                select(Offer)
                .where(Offer.is_active == True)
                .order_by(Offer.created_at.desc())
            )
        ).scalars().all()
    )


async def get_all_offers(session):
    return list(
        (await session.execute(select(Offer).order_by(Offer.created_at.desc()))).scalars().all()
    )


async def get_offer_by_id(session, oid):
    return (await session.execute(select(Offer).where(Offer.id == oid))).scalar_one_or_none()


async def toggle_offer_active(session, oid):
    o = await get_offer_by_id(session, oid)
    if not o:
        return None

    o.is_active = not o.is_active
    await session.commit()
    await session.refresh(o)
    return o


async def pin_offer_for_coins(session, user, offer_id):
    offer = await get_offer_by_id(session, offer_id)
    if not offer:
        return False, "\u041e\u0444\u0444\u0435\u0440 \u043d\u0435 \u043d\u0430\u0439\u0434\u0435\u043d."

    cost = to_decimal(PIN_OFFER_COST)
    if user.balance < cost:
        return False, "\u041d\u0435\u0445\u0432\u0430\u0442\u0430\u0435\u0442 \u043c\u043e\u043d\u0435\u0442."

    user.balance -= cost
    offer.created_at = datetime.utcnow()
    await session.commit()
    return True, f"\u041e\u0444\u0444\u0435\u0440 #{offer.id} \u043f\u043e\u0434\u043d\u044f\u0442 \u0437\u0430 {cost} \u043c\u043e\u043d\u0435\u0442."


async def start_offer_participation(session, user, offer):
    existing = (
```

```
        await session.execute(
            select(OfferParticipation).where(
                OfferParticipation.user_id == user.id,
                OfferParticipation.offer_id == offer.id,
            )
        )
    ).scalar_one_or_none()

    if existing:
        return False, "\u0423\u0436\u0435 \u0443\u0447\u0430\u0441\u0442\u0432\u0443\u0435\u0442
\u0435."

    session.add(
        OfferParticipation(
            user_id=user.id,
            offer_id=offer.id,
            status="started",
            reward_given=OFFER_STEP_1_REWARD,
        )
    )
    user.balance += OFFER_STEP_1_REWARD
    await session.commit()
    return True, f"+{OFFER_STEP_1_REWARD} \u043c\u043e\u043d\u0435\u0442."


async def verify_offer_subscription(session, user, offer, is_sub):
    part = (
        await session.execute(
            select(OfferParticipation).where(
                OfferParticipation.user_id == user.id,
                OfferParticipation.offer_id == offer.id,
            )
        )
    ).scalar_one_or_none()

    if not part:
        return False, "\u0421\u043d\u0430\u0447\u0430\u043b\u0430 \u043d\u0430\u0447\u043d\u0438
\u0442\u0435."

    part.checked_at = datetime.utcnow()

    if is_sub:
        if part.status == "verified":
            return True, "\u0423\u0436\u0435 \u043f\u043e\u0434\u0442\u0432\u0435\u0440\u0436\u0434
434\u0435\u043d\u043e."

        part.status = "verified"
        await session.commit()
        return True, "\u041f\u043e\u0434\u043f\u0438\u0441\u043a\u0430 OK."

    if part.status == "verified":
        user.balance = max(Decimal("0"), to_decimal(user.balance) - OFFER_PENALTY)
        part.status = "penalized"
        await session.commit()
        return False, f"\u0428\u0442\u0440\u0430\u0444 {OFFER_PENALTY}."

    await session.commit()
    return False, "\u041d\u0435 \u043f\u043e\u0434\u0442\u0432\u0435\u0440\u0436\u0434\u0435\u043
3d\u043e."


async def get_users_without_offer(session, offer_id):
    sub = select(OfferParticipation.user_id).where(OfferParticipation.offer_id == offer_id)
    return list(
        (
            await session.execute(
                select(User).where(
                    User.agreed_to_rules == True,
                    User.id.notin_(sub),
                )
```

```
            )
        ).scalars().all()
    )


# ========== CONTENT ==========

async def save_video(session, uploader, file_id, file_unique_id, duration, file_size):
    exists = (
        await session.execute(
            select(Video).where(Video.telegram_file_unique_id == file_unique_id)
        )
    ).scalar_one_or_none()
    if exists:
        return None

    v = Video(
        uploader_user_id=uploader.id,
        content_type="video",
        telegram_file_id=file_id,
        telegram_file_unique_id=file_unique_id,
        duration_seconds=duration,
        file_size=file_size,
        status="pending",
    )
    session.add(v)
    await session.commit()
    await session.refresh(v)
    return v


async def save_photo(session, uploader, file_id, file_unique_id, file_size):
    exists = (
        await session.execute(
            select(Video).where(Video.telegram_file_unique_id == file_unique_id)
        )
    ).scalar_one_or_none()
    if exists:
        return None

    p = Video(
        uploader_user_id=uploader.id,
        content_type="photo",
        telegram_file_id=file_id,
        telegram_file_unique_id=file_unique_id,
        file_size=file_size,
        status="pending",
    )
    session.add(p)
    await session.commit()
    await session.refresh(p)
    return p


async def bump_video_for_coins(session, user, video_id):
    video = (
        await session.execute(
            select(Video).where(
                Video.id == video_id,
                Video.uploader_user_id == user.id,
            )
        )
    ).scalar_one_or_none()

    if not video:
        return False, "\u0412\u0438\u0434\u0435\u043e \u043d\u0435 \u043d\u0430\u0439\u0434\u0435\u043
5\u043d\u043e."

    cost = to_decimal(BUMP_VIDEO_COST)
    if user.balance < cost:
```

```
        return False, "\u041d\u0435\u0445\u0432\u0430\u0442\u0430\u0435\u0442 \u043c\u043e\u043d
\u0435\u0442."

    user.balance -= cost
    video.created_at = datetime.utcnow()
    await session.commit()
    return True, f"\u0412\u0438\u0434\u0435\u043e #{video.id} \u043f\u043e\u0434\u043d\u044f\u04
42\u043e \u0437\u0430 {cost} \u043c\u043e\u043d\u0435\u0442."


# ========== MODERATION ==========

async def get_next_pending_video(session):
    return (
        await session.execute(
            select(Video)
            .where(Video.status == "pending")
            .order_by(Video.created_at)
            .limit(1)
        )
    ).scalar_one_or_none()


async def approve_video(session, video_id):
    v = (await session.execute(select(Video).where(Video.id == video_id))).scalar_one_or_none()
    if not v or v.status != "pending":
        return v

    v.status = "approved"
    v.rejection_reason = None

    up = (
        await session.execute(select(User).where(User.id == v.uploader_user_id))
    ).scalar_one_or_none()
    if up:
        reward = PHOTO_UPLOAD_REWARD if v.content_type == "photo" else to_decimal(UPLOAD_REWARD)
        up.balance += reward

    await session.commit()
    await session.refresh(v)
    return v


async def approve_all_pending(session):
    vids = list(
        (await session.execute(select(Video).where(Video.status == "pending"))).scalars().all()
    )

    for v in vids:
        v.status = "approved"
        v.rejection_reason = None

        up = (
            await session.execute(select(User).where(User.id == v.uploader_user_id))
        ).scalar_one_or_none()
        if up:
            reward = PHOTO_UPLOAD_REWARD if v.content_type == "photo" else to_decimal(UPLOAD_REW
ARD)
            up.balance += reward

    await session.commit()
    return len(vids)


async def reject_video(session, video_id, reason):
    v = (await session.execute(select(Video).where(Video.id == video_id))).scalar_one_or_none()
    if not v:
        return None

    v.status = "rejected"
```

```
    v.rejection_reason = reason
    await session.commit()
    await session.refresh(v)
    return v


async def count_pending_videos(session):
    return (
        await session.execute(select(func.count(Video.id)).where(Video.status == "pending"))
    ).scalar_one()


async def count_approved_videos(session):
    return (
        await session.execute(select(func.count(Video.id)).where(Video.status == "approved"))
    ).scalar_one()


async def count_rejected_videos(session):
    return (
        await session.execute(select(func.count(Video.id)).where(Video.status == "rejected"))
    ).scalar_one()


# ========== WATCH VIDEO ==========

async def get_video_stats_for_user(session, user):
    total = (
        await session.execute(
            select(func.count(Video.id)).where(
                Video.status == "approved",
                Video.content_type == "video",
            )
        )
    ).scalar_one()

    wsub = select(VideoView.video_id).where(VideoView.user_id == user.id)

    avail = (
        await session.execute(
            select(func.count(Video.id)).where(
                Video.status == "approved",
                Video.content_type == "video",
                Video.uploader_user_id != user.id,
                Video.id.notin_(wsub),
            )
        )
    ).scalar_one()

    return {
        "total_approved": total,
        "available": avail,
    }


async def get_random_video_for_user(session, user):
    wsub = select(VideoView.video_id).where(VideoView.user_id == user.id)
    return (
        await session.execute(
            select(Video).where(
                Video.status == "approved",
                Video.content_type == "video",
                Video.uploader_user_id != user.id,
                Video.id.notin_(wsub),
            ).order_by(func.random()).limit(1)
        )
    ).scalar_one_or_none()


async def record_view_and_charge(session, user, video):
```

```
    cost = get_watch_cost_for_user(user)

    if user.balance < cost:
        return False

    exists = (
        await session.execute(
            select(VideoView).where(
                VideoView.user_id == user.id,
                VideoView.video_id == video.id,
            )
        )
    ).scalar_one_or_none()

    if exists:
        return False

    session.add(VideoView(user_id=user.id, video_id=video.id))
    user.balance -= cost
    await session.commit()
    return True


# ========== WATCH PHOTO ==========

async def count_photo_views_last_4h(session, user_id):
    since = datetime.utcnow() - timedelta(hours=4)
    return (
        await session.execute(
            select(func.count(VideoView.id))
            .join(Video)
            .where(
                VideoView.user_id == user_id,
                Video.content_type == "photo",
                VideoView.watched_at >= since,
            )
        )
    ).scalar_one()


async def get_random_photo_for_user(session, user):
    wsub = select(VideoView.video_id).where(VideoView.user_id == user.id)
    return (
        await session.execute(
            select(Video).where(
                Video.status == "approved",
                Video.content_type == "photo",
                Video.uploader_user_id != user.id,
                Video.id.notin_(wsub),
            ).order_by(func.random()).limit(1)
        )
    ).scalar_one_or_none()


async def record_photo_view(session, user, photo):
    exists = (
        await session.execute(
            select(VideoView).where(
                VideoView.user_id == user.id,
                VideoView.video_id == photo.id,
            )
        )
    ).scalar_one_or_none()

    if exists:
        return False

    session.add(VideoView(user_id=user.id, video_id=photo.id))
    await session.commit()
    return True
```

```
# ========== RATING ==========

async def rate_video(session, user_id, video_id, rating):
    existing = (
        await session.execute(
            select(VideoRating).where(
                VideoRating.user_id == user_id,
                VideoRating.video_id == video_id,
            )
        )
    ).scalar_one_or_none()

    if existing:
        existing.rating = rating
    else:
        session.add(VideoRating(user_id=user_id, video_id=video_id, rating=rating))

    await session.commit()


# ========== COMMENTS ==========

async def can_user_comment(session, user_id):
    now = datetime.utcnow()
    window_start = now - timedelta(minutes=10)

    recent_count = (
        await session.execute(
            select(func.count(Comment.id)).where(
                Comment.user_id == user_id,
                Comment.created_at >= window_start,
            )
        )
    ).scalar_one()

    last_comment = (
        await session.execute(
            select(Comment)
            .where(Comment.user_id == user_id)
            .order_by(Comment.created_at.desc())
            .limit(1)
        )
    ).scalar_one_or_none()

    if recent_count >= COMMENTS_PER_10_MIN:
        return False, "\u0421\u043b\u0438\u0448\u043a\u043e\u043c \u043c\u043d\u043e\u0433\u043e
 \u043a\u043e\u043c\u043c\u0435\u043d\u0442\u0430\u0440\u0438\u0435\u0432. \u041f\u043e\u0434\u0
43e\u0436\u0434\u0438\u0442\u0435."

    if last_comment:
        delta = (now - last_comment.created_at).total_seconds()
        if delta < COMMENT_MIN_INTERVAL_SEC:
            wait_sec = int(COMMENT_MIN_INTERVAL_SEC - delta)
            return False, f"\u041f\u043e\u0434\u043e\u0436\u0434\u0438\u0442\u0435 {wait_sec} \u
0441\u0435\u043a."

    return True, ""


async def add_comment(session, user_id, video_id, text):
    c = Comment(user_id=user_id, video_id=video_id, text=text[:500])
    session.add(c)
    await session.commit()
    await session.refresh(c)
    return c


async def get_video_comments(session, video_id, limit=10):
```

```
    stmt = (
        select(Comment, User.username, User.first_name)
        .join(User, User.id == Comment.user_id)
        .where(Comment.video_id == video_id)
        .order_by(Comment.created_at.desc())
        .limit(limit)
    )
    rows = (await session.execute(stmt)).all()

    result = []
    for comment, uname, fname in rows:
        name = f"@{uname}" if uname else (fname or "Anon")
        result.append({
            "id": comment.id,
            "text": comment.text,
            "author": name,
            "at": comment.created_at,
        })

    return result


async def count_user_comments_today(session, user_id):
    today_start = datetime.utcnow().replace(hour=0, minute=0, second=0, microsecond=0)
    return (
        await session.execute(
            select(func.count(Comment.id)).where(
                Comment.user_id == user_id,
                Comment.created_at >= today_start,
            )
        )
    ).scalar_one()


# ========== REACTIONS ==========

async def add_reaction(session, user_id, video_id, reaction_type):
    existing = (
        await session.execute(
            select(ContentReaction).where(
                ContentReaction.user_id == user_id,
                ContentReaction.video_id == video_id,
            )
        )
    ).scalar_one_or_none()

    if existing:
        existing.reaction_type = reaction_type
    else:
        session.add(
            ContentReaction(
                user_id=user_id,
                video_id=video_id,
                reaction_type=reaction_type,
            )
        )

    await session.commit()


async def get_reaction_counts(session, video_id):
    stmt = (
        select(ContentReaction.reaction_type, func.count(ContentReaction.id))
        .where(ContentReaction.video_id == video_id)
        .group_by(ContentReaction.reaction_type)
    )
    rows = (await session.execute(stmt)).all()
    return {r: c for r, c in rows}
```

```
# ========== GAMES ==========

async def play_lootbox(session, user):
    cost = to_decimal(LOOTBOX_COST)
    if user.balance < cost:
        return False, 0, "\u041d\u0435\u0434\u043e\u0441\u0442\u0430\u0442\u043e\u0447\u043d\u043
3e \u043c\u043e\u043d\u0435\u0442."

    user.balance -= cost

    roll = random.random()
    cumul = 0
    reward = 1

    for prob, coins in LOOTBOX_REWARDS:
        cumul += prob
        if roll <= cumul:
            reward = coins
            break

    user.balance += to_decimal(reward)

    session.add(
        GameHistory(
            user_id=user.id,
            game_type="lootbox",
            bet=cost,
            result=to_decimal(reward),
            details=f"won {reward}",
        )
    )
    await session.commit()
    await session.refresh(user)
    return True, reward, ""


async def play_dice(session, user, bet):
    bet_d = to_decimal(bet)

    if bet < DICE_MIN_BET or bet > DICE_MAX_BET:
        return False, 0, Decimal("0"), f"\u0421\u0442\u0430\u0432\u043a\u0430 {DICE_MIN_BET}-{DI
CE_MAX_BET}."
    if user.balance < bet_d:
        return False, 0, Decimal("0"), "\u041d\u0435\u0434\u043e\u0441\u0442\u0430\u0442\u043e\u
0447\u043d\u043e."

    user.balance -= bet_d
    roll = random.randint(1, 6)

    if roll >= 4:
        win = bet_d * 2
        user.balance += win
    else:
        win = Decimal("0")

    session.add(
        GameHistory(
            user_id=user.id,
            game_type="dice",
            bet=bet_d,
            result=win,
            details=f"roll={roll}",
        )
    )
    await session.commit()
    await session.refresh(user)
    return True, roll, win, ""


async def play_coinflip(session, user, bet):
```

```
    bet_d = to_decimal(bet)

    if bet < 1 or bet > DICE_MAX_BET:
        return False, "", Decimal("0"), "\u0421\u0442\u0430\u0432\u043a\u0430 1-50."
    if user.balance < bet_d:
        return False, "", Decimal("0"), "\u041d\u0435\u0434\u043e\u0441\u0442\u0430\u0442\u043e\
u0447\u043d\u043e."

    user.balance -= bet_d
    side = random.choice(["heads", "tails"])

    if side == "heads":
        win = bet_d * 2
        user.balance += win
    else:
        win = Decimal("0")

    session.add(
        GameHistory(
            user_id=user.id,
            game_type="coinflip",
            bet=bet_d,
            result=win,
            details=side,
        )
    )
    await session.commit()
    await session.refresh(user)
    return True, side, win, ""


async def play_guess(session, user, guess, bet):
    bet_d = to_decimal(bet)

    if guess < 1 or guess > 10:
        return False, 0, Decimal("0"), "\u0427\u0438\u0441\u043b\u043e 1-10."
    if bet < 1 or bet > DICE_MAX_BET:
        return False, 0, Decimal("0"), "\u0421\u0442\u0430\u0432\u043a\u0430 1-50."
    if user.balance < bet_d:
        return False, 0, Decimal("0"), "\u041d\u0435\u0434\u043e\u0441\u0442\u0430\u0442\u043e\u
0447\u043d\u043e."

    user.balance -= bet_d
    answer = random.randint(1, 10)

    if guess == answer:
        win = bet_d * 5
        user.balance += win
    else:
        win = Decimal("0")

    session.add(
        GameHistory(
            user_id=user.id,
            game_type="guess",
            bet=bet_d,
            result=win,
            details=f"g={guess},a={answer}",
        )
    )
    await session.commit()
    await session.refresh(user)
    return True, answer, win, ""


# ========== QUESTS ==========

async def ensure_daily_quests(session, user_id, user=None):
    today = datetime.utcnow().date()
```

```
    existing = list(
        (
            await session.execute(
                select(DailyQuestProgress).where(
                    DailyQuestProgress.user_id == user_id,
                    DailyQuestProgress.quest_date == today,
                )
            )
        ).scalars().all()
    )
    if existing:
        return existing

    quests = []
    all_quests = list(DAILY_QUESTS)

    if user and is_vip(user):
        all_quests.extend(PREMIUM_DAILY_QUESTS)

    for q in all_quests:
        qp = DailyQuestProgress(
            user_id=user_id,
            quest_type=q["type"],
            quest_date=today,
            progress=0,
            target=q["target"],
            reward=to_decimal(q["reward"]),
            completed=False,
            reward_claimed=False,
        )
        session.add(qp)
        quests.append(qp)

    await session.commit()
    for qp in quests:
        await session.refresh(qp)

    return quests


async def increment_quest(session, user_id, quest_type):
    today = datetime.utcnow().date()
    q_all = list(
        (
            await session.execute(
                select(DailyQuestProgress).where(
                    DailyQuestProgress.user_id == user_id,
                    DailyQuestProgress.quest_type == quest_type,
                    DailyQuestProgress.quest_date == today,
                    DailyQuestProgress.completed == False,
                )
            )
        ).scalars().all()
    )

    for q in q_all:
        q.progress += 1
        if q.progress >= q.target:
            q.completed = True

    await session.commit()


async def claim_quest_reward(session, user, quest_id):
    q = (
        await session.execute(
            select(DailyQuestProgress).where(
                DailyQuestProgress.id == quest_id,
                DailyQuestProgress.user_id == user.id,
                DailyQuestProgress.completed == True,
```

```
                DailyQuestProgress.reward_claimed == False,
            )
        )
    ).scalar_one_or_none()

    if not q:
        return False, "\u041d\u0435\u0442 \u043d\u0430\u0433\u0440\u0430\u0434\u0430\u0434\u044b."

    user.balance += q.reward
    q.reward_claimed = True
    await session.commit()
    await session.refresh(user)
    return True, f"+{q.reward}. \u0411\u0430\u043b\u0430\u043d\u0441: {user.balance}"


# ========== TOPS ==========

async def get_top_uploaders(session, limit=10):
    stmt = (
        select(
            User.username,
            User.first_name,
            User.telegram_id,
            func.count(Video.id).label("cnt"),
        )
        .join(Video, Video.uploader_user_id == User.id)
        .where(Video.status == "approved")
        .group_by(User.id)
        .order_by(func.count(Video.id).desc())
        .limit(limit)
    )
    return (await session.execute(stmt)).all()


async def get_top_viewers(session, limit=10):
    stmt = (
        select(
            User.username,
            User.first_name,
            User.telegram_id,
            func.count(VideoView.id).label("cnt"),
        )
        .join(VideoView, VideoView.user_id == User.id)
        .group_by(User.id)
        .order_by(func.count(VideoView.id).desc())
        .limit(limit)
    )
    return (await session.execute(stmt)).all()


async def get_top_by_level(session, limit=10):
    stmt = select(User).where(User.agreed_to_rules == True).order_by(User.xp.desc()).limit(limit
)
    return list((await session.execute(stmt)).scalars().all())


async def get_top_richest(session, limit=10):
    stmt = select(User).where(User.agreed_to_rules == True).order_by(User.balance.desc()).limit(
limit)
    return list((await session.execute(stmt)).scalars().all())


# ========== WEEKLY REWARDS ==========

async def reward_weekly_top_users(session):
    rewarded = []
    richest = await get_top_richest(session, limit=3)

    rewards = [
        to_decimal(WEEKLY_TOP1_REWARD),
```

```
        to_decimal(WEEKLY_TOP2_REWARD),
        to_decimal(WEEKLY_TOP3_REWARD),
    ]

    for idx, user in enumerate(richest):
        reward = rewards[idx]
        user.balance += reward
        rewarded.append((user, reward))

    await session.commit()
    return rewarded


# ========== ADMIN STATS ==========

async def get_admin_extended_stats(session):
    users_count = (await session.execute(select(func.count(User.id)))).scalar_one()

    vip_count = (
        await session.execute(
            select(func.count(User.id)).where(
                User.vip_until.is_not(None),
                User.vip_until > datetime.utcnow(),
            )
        )
    ).scalar_one()

    comments_count = (await session.execute(select(func.count(Comment.id)))).scalar_one()
    reactions_count = (await session.execute(select(func.count(ContentReaction.id)))).scalar_one
()
    games_count = (await session.execute(select(func.count(GameHistory.id)))).scalar_one()
    offers_count = (await session.execute(select(func.count(Offer.id)))).scalar_one()

    return {
        "users": users_count,
        "vip": vip_count,
        "comments": comments_count,
        "reactions": reactions_count,
        "games": games_count,
        "offers": offers_count,
    }

================================================================================
FILE: user_handlers.py
================================================================================
from decimal import Decimal

from aiogram import Router, F
from aiogram.filters import CommandStart, CommandObject
from aiogram.types import Message, CallbackQuery, LabeledPrice, PreCheckoutQuery
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup

from app.config import (
    ADMINS,
    WATCH_COST,
    STARS_PACKAGES,
    STARS_TO_COINS_RATE,
    REACTION_TYPES,
    XP_PER_WATCH,
    XP_PER_UPLOAD,
    XP_PER_RATING,
    XP_PER_COMMENT,
    XP_PER_REACTION,
    XP_PER_GAME,
)
from app.db import async_session
from app.services import (
    get_or_create_user,
    agree_to_rules,
```

```
    get_user,
    save_video,
    save_photo,
    get_random_video_for_user,
    get_random_photo_for_user,
    record_view_and_charge,
    record_photo_view,
    count_photo_views_last_4h,
    rate_video,
    claim_daily_bonus,
    count_referrals,
    create_payment,
    create_custom_payment,
    create_vip_payment,
    apply_successful_payment,
    get_active_offers,
    get_offer_by_id,
    start_offer_participation,
    verify_offer_subscription,
    FREE_PHOTO_LIMIT_PER_4H,
    add_xp,
    calc_level_info,
    is_vip,
    ensure_daily_quests,
    increment_quest,
    claim_quest_reward,
    get_top_uploaders,
    get_top_viewers,
    get_top_by_level,
    get_top_richest,
    play_lootbox,
    play_dice,
    play_coinflip,
    play_guess,
    add_comment,
    get_video_comments,
    add_reaction,
    get_reaction_counts,
)
from app.keyboards import (
    rules_keyboard,
    main_menu,
    video_rating_keyboard,
    photo_actions_keyboard,
    watch_choice_keyboard,
    admin_center_keyboard,
    buy_coins_keyboard,
    vip_buy_keyboard,
    offers_list_keyboard,
    offer_view_keyboard,
    games_menu_keyboard,
    tops_menu_keyboard,
    quests_keyboard,
    reaction_menu_keyboard,
    BTN_WATCH,
    BTN_UPLOAD,
    BTN_PROFILE,
    BTN_BUY,
    BTN_OFFERS,
    BTN_REFERRALS,
    BTN_BONUS,
    BTN_ADMIN,
    BTN_GAMES,
    BTN_TOPS,
    BTN_QUESTS,
    BTN_VIP,
    BTN_LEVEL,
)
from app.logger import get_logger, log_info, log_warning, log_exception
```

```
logger = get_logger(__name__)
router = Router()

RULES_TEXT = (
    "\u26a0\ufe0f <b>\u041f\u0440\u0430\u0432\u0438\u043b\u0430</b>\n\n"
    "1. \u0411\u043e\u0442 \u0441\u043e\u0434\u0435\u0440\u0436\u0438\u0442 \u043a\u043e\u043d\u0442\u0435\u043d\u0442 18+.\n"
    "2. \u0418\u0441\u043f\u043e\u043b\u044c\u0437\u0443\u044f \u0431\u043e\u0442, \u0432\u044b \u043f\u043e\u0434\u0442\u0432\u0435\u0440\u0436\u0434\u0430\u0435\u0442\u0435 18+.\n"
    "3. \u0417\u0430\u043f\u0440\u0435\u0449\u0435\u043d\u043e CP.\n"
    "4. \u0417\u0430\u043f\u0440\u0435\u0449\u0451\u043d \u043a\u043e\u043d\u0442\u0435\u043d\u0442 \u0441 \u043d\u0430\u0441\u0438\u043b\u0438\u0435\u043c.\n"
    "5. \u0410\u0434\u043c\u0438\u043d\u0438\u0441\u0442\u0440\u0430\u0446\u0438\u044f \u043c\u043e\u0436\u0435\u0442 \u043e\u0433\u0440\u0430\u043d\u0438\u0447\u0438\u0442\u044c \u0434\u043e\u0441\u0442\u0443\u043f.\n\n"
    "\u041d\u0430\u0436\u0438\u043c\u0430\u044f \u043a\u043d\u043e\u043f\u043a\u0438 \u043d\u0438\u0436\u0435."
)


class CustomPayState(StatesGroup):
    waiting_amount = State()


class GameState(StatesGroup):
    waiting_dice_bet = State()
    waiting_coin_bet = State()
    waiting_guess_number = State()
    waiting_guess_bet = State()


class CommentState(StatesGroup):
    waiting_comment_text = State()


def is_any_admin(telegram_id: int, user_obj=None) -> bool:
    if telegram_id in ADMINS:
        return True
    if user_obj and user_obj.is_admin:
        return True
    return False


def is_super_admin(telegram_id: int) -> bool:
    return telegram_id in ADMINS


def extract_channel_id(channel_url: str) -> str:
    url = channel_url.strip().rstrip("/")
    if url.startswith("https://t.me/"):
        return f"@{url.replace('https://t.me/', '', 1)}"
    if url.startswith("http://t.me/"):
        return f"@{url.replace('http://t.me/', '', 1)}"
    if url.startswith("t.me/"):
        return f"@{url.replace('t.me/', '', 1)}"
    if url.startswith("@"):
        return url
    return f"@{url}"


async def safe_edit_or_answer(callback: CallbackQuery, text: str, reply_markup=None):
    try:
        await callback.message.edit_text(text, parse_mode="HTML", reply_markup=reply_markup)
    except Exception:
        await callback.message.answer(text, parse_mode="HTML", reply_markup=reply_markup)


async def get_user_or_reply(
    event: Message | CallbackQuery,
    telegram_id: int,
```

```
    require_rules: bool = True,
):
    async with async_session() as session:
        user = await get_user(session, telegram_id)

    if not user:
        if isinstance(event, CallbackQuery):
            await event.answer("/start", show_alert=True)
        else:
            await event.answer("/start")
        return None

    if require_rules and not user.agreed_to_rules:
        if isinstance(event, CallbackQuery):
            await event.answer(
                "\u0421\u043d\u0430\u0447\u0430\u043b\u0430 \u043f\u0440\u0438\u043c\u0438\u0442\
\u0435 \u043f\u0440\u0430\u0432\u0438\u043b\u0430",
                show_alert=True,
            )
        else:
            await event.answer(
                "\u0421\u043d\u0430\u0447\u0430\u043b\u0430 \u043f\u0440\u0438\u043c\u0438\u0442\
\u0435 \u043f\u0440\u0430\u0432\u0438\u043b\u0430 \u0447\u0435\u0440\u0435\u0437 /start"
            )
        return None

    return user


def format_level_text(user) -> str:
    lvl, current_xp, need_xp = calc_level_info(user.xp)
    vip_text = "\n\U0001f48e VIP: \u0430\u043a\u0442\u0438\u0432\u0435\u043d" if is_vip(user) el
se ""
    return (
        f"\U0001f4c8 <b>\u0423\u0440\u043e\u0432\u0435\u043d\u044c</b>\n\n"
        f"\U0001f3c5 \u0423\u0440\u043e\u0432\u0435\u043d\u044c: <b>{lvl}</b>\n"
        f"XP: <b>{user.xp}</b>\n"
        f"\u041f\u0440\u043e\u0433\u0440\u0435\u0441\u0441: <b>{current_xp}/{need_xp}</b>{vip_te
xt}"
    )


@router.message(CommandStart())
async def cmd_start(message: Message, command: CommandObject, state: FSMContext):
    if not message.from_user:
        return

    await state.clear()
    referral_code = command.args.strip() if command and command.args else None

    try:
        async with async_session() as session:
            user, is_new = await get_or_create_user(
                session,
                message.from_user.id,
                message.from_user.username,
                message.from_user.first_name,
                message.from_user.last_name,
                referral_code,
            )

            if not user.agreed_to_rules:
                log_info(
                    logger,
                    "[redacted]",
                    tg_id=message.from_user.id,
                    is_new=is_new,
                    referral_code=referral_code,
                )
                await message.answer(
```

```
                    RULES_TEXT,
                    parse_mode="HTML",
                    reply_markup=rules_keyboard(),
                )
                return

            await ensure_daily_quests(session, user.id, user)
            admin_flag = is_any_admin(message.from_user.id, user)

            log_info(
                logger,
                "[redacted]",
                tg_id=message.from_user.id,
                username=message.from_user.username,
                is_new=is_new,
                balance=str(user.balance),
                is_admin=admin_flag,
            )

            await message.answer(
                f"\U0001f44b \u0421 \u0432\u043e\u0437\u0432\u0440\u0430\u0449\u0435\u043d\u0438
\u0435\u043c!\n"
                f"\U0001f4b0 \u0411\u0430\u043b\u0430\u043d\u0441: <b>{user.balance}</b>",
                parse_mode="HTML",
                reply_markup=main_menu(is_admin=admin_flag),
            )
    except Exception:
        log_exception(
            logger,
            "[redacted] /start",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


@router.callback_query(F.data == "accept_rules")
async def cb_accept_rules(callback: CallbackQuery):
    if not callback.from_user:
        return

    try:
        async with async_session() as session:
            await agree_to_rules(session, callback.from_user.id)
            user = await get_user(session, callback.from_user.id)
            if not user:
                await callback.answer("/start", show_alert=True)
                return

            await ensure_daily_quests(session, user.id, user)
            admin_flag = is_any_admin(callback.from_user.id, user)

        log_info(
            logger,
            "[redacted]",
            tg_id=callback.from_user.id,
        )

        await callback.message.edit_text("\u2705 \u041f\u0440\u0430\u0432\u0438\u043b\u0430 \u04
3f\u0440\u0438\u043d\u044f\u0442\u044b.")
        await callback.message.answer(
            "\u0414\u043e\u0431\u0440\u043e \u043f\u043e\u0436\u0430\u043b\u043e\u0432\u0430\u04
42\u044c! \u0421\u0442\u0430\u0440\u0442\u043e\u0432\u044b\u0439 \u0431\u0430\u043b\u0430\u043d\
u0441: <b>2</b>",
            parse_mode="HTML",
            reply_markup=main_menu(is_admin=admin_flag),
        )
        await callback.answer()
    except Exception:
        log_exception(
            logger,
```

```
            "[redacted]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.message(F.text == BTN_BONUS)
async def daily_bonus(message: Message):
    if not message.from_user:
        return

    try:
        user = await get_user_or_reply(message, message.from_user.id)
        if not user:
            return

        async with async_session() as session:
            success, msg = await claim_daily_bonus(session, message.from_user.id)

        log_info(
            logger,
            "[redacted]",
            tg_id=message.from_user.id,
            success=success,
            result=msg,
        )

        await message.answer(f"\U0001f3c6 {msg}" if success else f"\u23f3 {msg}")
    except Exception:
        log_exception(
            logger,
            "[redacted]",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


@router.message(F.text == BTN_PROFILE)
async def show_profile(message: Message):
    if not message.from_user:
        return

    try:
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not user:
                await message.answer("/start")
                return

            admin_flag = is_any_admin(message.from_user.id, user)
            role = "\u0410\u0434\u043c\u0438\u043d" if admin_flag else "\u041f\u043e\u043b\u044c
\u0437\u043e\u0432\u0430\u0442\u0435\u043b\u044c"

            vip_text = ""
            if is_vip(user) and user.vip_until:
                vip_text = "\n\U0001f48e VIP \u0434\u043e: <b>" + user.vip_until.strftime("%d.%m
.%Y") + "</b>"

            text = (
                f"\U0001f464 <b>\u041f\u0440\u043e\u0444\u0438\u043b\u044c</b>\n\n"
                f"ID: <code>{user.telegram_id}</code>\n"
                f"\U0001f4b0 \u0411\u0430\u043b\u0430\u043d\u0441: <b>{user.balance}</b>\n"
                f"\U0001f4c8 XP: <b>{user.xp}</b>\n"
                f"\U0001f3c5 \u0423\u0440\u043e\u0432\u0435\u043d\u044c: <b>{user.level}</b>\n"
                f"\u0420\u043e\u043b\u044c: {role}{vip_text}"
            )

        log_info(
            logger,
            "[redacted]",
```

```
            tg_id=message.from_user.id,
            balance=str(user.balance),
            xp=user.xp,
            level=user.level,
        )

        await message.answer(text, parse_mode="HTML")
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


@router.message(F.text == BTN_LEVEL)
async def level_info(message: Message):
    if not message.from_user:
        return

    try:
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not user:
                await message.answer("/start")
                return

        log_info(
            logger,
            "[illegible]",
            tg_id=message.from_user.id,
            xp=user.xp,
            level=user.level,
        )

        await message.answer(format_level_text(user), parse_mode="HTML")
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


@router.message(F.text == BTN_REFERRALS)
async def referrals_info(message: Message):
    if not message.from_user:
        return

    try:
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not user:
                await message.answer("/start")
                return

            referrals_count = await count_referrals(session, user.id)
            bot_username = (await message.bot.get_me()).username
            referral_link = f"https://t.me/{bot_username}?start={user.referral_code}"

            text = (
                f"\U0001f465 <b>\u041f\u0440\u0438\u0433\u043b\u0430\u0441\u0438 \u0434\u0440\u0443\u0433\u0430!</b>\n\n"
                f"\U0001f517 \u0422\u0432\u043e\u044f \u0441\u0441\u044b\u043b\u043a\u0430:\n<code>{referral_link}</code>\n\n"
                f"\U0001f464 \u041f\u0440\u0438\u0441\u043e\u0435\u0434\u0438\u043d\u0438\u043b\u043e\u0441\u044c: <b>{referrals_count}</b>\n"
                f"\U0001f4b0 \u0417\u0430\u0440\u0430\u0431\u043e\u0442\u043e\u043a\u043e: <b>{u
```

```
ser.referral_earnings}</b>"
            )

        log_info(
            logger,
            "[redacted]",
            tg_id=message.from_user.id,
            referrals_count=referrals_count,
            referral_earnings=str(user.referral_earnings),
        )

        await message.answer(text, parse_mode="HTML")
    except Exception:
        log_exception(
            logger,
            "[redacted]",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


@router.message(F.text == BTN_BUY)
async def buy_coins(message: Message):
    log_info(
        logger,
        "[redacted]",
        tg_id=message.from_user.id if message.from_user else None,
    )
    await message.answer(
        "\U0001f48e <b>\u041f\u043e\u043a\u0443\u043f\u043a\u0430 \u043c\u043e\u043d\u0435\u0442
</b>\n\n"
        f"\u041a\u0443\u0440\u0441: 1 Star = {STARS_TO_COINS_RATE} \u043c\u043e\u043d\u0435\u044
2\n\n"
        "\u0412\u044b\u0431\u0435\u0440\u0438\u0442\u0435 \u043f\u0430\u043a\u0435\u0442 \u0438\
u043b\u0438 \u0432\u0432\u0435\u0434\u0438\u0442\u0435 \u0441\u0432\u043e\u044e \u0441\u0443\u04
3c\u043c\u0443:",
        parse_mode="HTML",
        reply_markup=buy_coins_keyboard(),
    )


@router.message(F.text == BTN_VIP)
async def vip_info(message: Message):
    if not message.from_user:
        return

    try:
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not user:
                await message.answer("/start")
                return

            status = "\u2705 \u0410\u043a\u0442\u0438\u0432\u0435\u043d" if is_vip(user) else "\
u274c \u041d\u0435\u0442"
            text = (
                "\U0001f48e <b>VIP</b>\n\n"
                f"\u0421\u0442\u0430\u0442\u0443\u0441: {status}\n"
                "\u0411\u043e\u043d\u0443\u0441\u044b:\n"
                "\u2022 x2 \u043a \u0435\u0436\u0435\u0434\u043d\u0435\u0432\u043d\u043e\u043c\u0
0443 \u0431\u043e\u043d\u0443\u0441\u0443\n"
                "\u2022 VIP \u0432 \u043f\u0440\u043e\u0444\u0438\u043b\u0435\n"
                "\u2022 \u0431\u0435\u0437\u043b\u0438\u043c\u0438\u0442 \u0444\u043e\u0442\u043
e\n\n"
                "\u0426\u0435\u043d\u0430: 50 Stars / 30 \u0434\u043d\u0435\u0439"
            )

        log_info(
            logger,
            "[redacted] VIP",
```

```
            tg_id=message.from_user.id,
            vip_active=is_vip(user),
        )

        await message.answer(text, parse_mode="HTML", reply_markup=vip_buy_keyboard())
    except Exception:
        log_exception(
            logger,
            "[redacted] VIP",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


@router.callback_query(F.data == "buy_vip")
async def buy_vip(callback: CallbackQuery):
    if not callback.from_user:
        return

    try:
        async with async_session() as session:
            user = await get_user(session, callback.from_user.id)
            if not user:
                await callback.answer("/start", show_alert=True)
                return

            payment = await create_vip_payment(session, user)

        log_info(
            logger,
            "[redacted] VIP-[redacted]",
            tg_id=callback.from_user.id,
            payload=payment.payload,
            stars_amount=payment.stars_amount,
        )

        await callback.message.answer_invoice(
            title="VIP 30 \u0434\u043d\u0435\u0439",
            description="\u0410\u043a\u0442\u0438\u0432\u0430\u0446\u0438\u044f VIP \u043d\u0430
 30 \u0434\u043d\u0435\u0439",
            payload=payment.payload,
            provider_token="",
            currency="XTR",
            prices=[LabeledPrice(label="VIP 30 \u0434\u043d\u0435\u0439", amount=50)],
        )
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[redacted] VIP-[redacted]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data.startswith("buy:"))
async def cb_buy_package(callback: CallbackQuery):
    if not callback.from_user:
        return

    package_key = callback.data.split(":", 1)[1]
    package = STARS_PACKAGES.get(package_key)
    if not package:
        await callback.answer("\u041d\u0435 \u043d\u0430\u0439\u0434\u0435\u043d", show_alert=Tr
ue)
        return

    try:
        async with async_session() as session:
            user = await get_user(session, callback.from_user.id)
```

```
            if not user:
                await callback.answer("/start", show_alert=True)
                return

            payment = await create_payment(session, user, package_key)
            if not payment:
                await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)
                return

        log_info(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id,
            package_key=package_key,
            payload=payment.payload,
            stars_amount=payment.stars_amount,
            coins_amount=str(payment.coins_amount),
        )

        await callback.message.answer_invoice(
            title=f"{package['coins']} \u043c\u043e\u043d\u0435\u0442",
            description=f"\u041f\u043e\u043f\u043e\u043b\u043d\u0435\u043d\u0438\u0435 \u043d\u0
430 {package['coins']} \u043c\u043e\u043d\u0435\u0442",
            payload=payment.payload,
            provider_token="",
            currency="XTR",
            prices=[LabeledPrice(label=f"{package['coins']} \u043c\u043e\u043d\u0435\u0442", amo
unt=package["stars"])],
        )
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
            package_key=package_key,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data == "buy_custom")
async def cb_buy_custom(callback: CallbackQuery, state: FSMContext):
    if not callback.from_user:
        return

    await state.set_state(CustomPayState.waiting_amount)

    log_info(
        logger,
        "[illegible] Stars",
        tg_id=callback.from_user.id,
    )

    await callback.message.answer(
        f"\U0001f4dd \u0412\u0432\u0435\u0434\u0438\u0442\u0435 \u043a\u043e\u043b-\u0432\u043e
\u0437\u0432\u0451\u0437\u0434 (Stars).\n"
        f"\u041a\u0443\u0440\u0441: 1 Star = {STARS_TO_COINS_RATE} \u043c\u043e\u043d\u0435\u0442
2\n\n"
        "\u041c\u0438\u043d: 1 Star\n/start \u0434\u043b\u044f \u043e\u0442\u043c\u0435\u043d\u0
44b"
    )
    await callback.answer()


@router.message(CustomPayState.waiting_amount)
async def custom_pay_amount(message: Message, state: FSMContext):
    if not message.from_user:
        return

    text = (message.text or "").strip()
```

```
    if not text.isdigit() or int(text) < 1:
        await message.answer("\u0412\u0432\u0435\u0434\u0438\u0442\u0435 \u0447\u0438\u0441\u043
b\u043e \u043e\u0442 1.")
        return

    stars = int(text)
    coins = Decimal(str(stars)) * Decimal(str(STARS_TO_COINS_RATE))

    try:
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not user:
                await message.answer("/start")
                await state.clear()
                return

            payment = await create_custom_payment(session, user, stars)

        log_info(
            logger,
            "[illegible]",
            tg_id=message.from_user.id,
            payload=payment.payload,
            stars_amount=stars,
            coins_amount=str(coins),
        )

        await message.answer_invoice(
            title=f"{coins} \u043c\u043e\u043d\u0435\u0442",
            description=f"{stars} Stars \u2192 {coins} \u043c\u043e\u043d\u0435\u0442",
            payload=payment.payload,
            provider_token="",
            currency="XTR",
            prices=[LabeledPrice(label=f"{coins} \u043c\u043e\u043d\u0435\u0442", amount=stars)]
,
        )
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=message.from_user.id if message.from_user else None,
            stars=stars,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")
    finally:
        await state.clear()


@router.pre_checkout_query()
async def pre_checkout(pq: PreCheckoutQuery):
    await pq.answer(ok=True)


@router.message(F.successful_payment)
async def successful_payment(message: Message):
    if not message.from_user or not message.successful_payment:
        return

    try:
        payload = message.successful_payment.invoice_payload

        async with async_session() as session:
            success, msg = await apply_successful_payment(session, payload)

        log_info(
            logger,
            "[illegible]",
            tg_id=message.from_user.id,
            payload=payload,
            success=success,
```

```
            result=msg,
        )

        await message.answer(f"\u2705 {msg}" if success else f"\u26a0\ufe0f {msg}")
    except Exception:
        log_exception(
            logger,
            "[illegible] [illegible] successful_payment",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430 \u043d\u0430\u0447\u0438\u0441
1\u043b\u0435\u043d\u0438\u044f.")


@router.message(F.text == BTN_OFFERS)
async def show_offers(message: Message):
    if not message.from_user:
        return

    try:
        async with async_session() as session:
            offers = await get_active_offers(session)
            if not offers:
                await message.answer("\u041e\u0444\u0444\u0435\u0440\u043e\u0432 \u043d\u0435\u0
442.")
                return

        log_info(
            logger,
            "[illegible] [illegible] [illegible]",
            tg_id=message.from_user.id,
            offers_count=len(offers),
        )

        await message.answer(
            "\U0001f381 <b>\u041e\u0444\u0444\u0435\u0440\u044b</b>",
            parse_mode="HTML",
            reply_markup=offers_list_keyboard(offers),
        )
    except Exception:
        log_exception(
            logger,
            "[illegible] [illegible] [illegible] [illegible] [illegible]",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


@router.callback_query(F.data.startswith("offer_open:"))
async def offer_open(callback: CallbackQuery):
    if not callback.from_user:
        return

    try:
        offer_id = int(callback.data.split(":")[1])

        async with async_session() as session:
            offer = await get_offer_by_id(session, offer_id)
            if not offer or not offer.is_active:
                await callback.answer("\u041d\u0435\u0434\u043e\u0441\u0442\u0443\u043f\u0435\u0
43d", show_alert=True)
                return

        text = (
            f"\U0001f381 <b>{offer.title}</b>\n\n{offer.description}\n\n"
            f"\U0001f517 {offer.channel_url}\n"
            f"\U0001f4b0 \u0412\u0441\u0435\u0433\u043e: <b>40</b>\n"
            f"\u26a0\ufe0f \u0428\u0442\u0440\u0430\u0444: <b>40</b>"
        )
```

```
        log_info(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id,
            offer_id=offer.id,
            title=offer.title,
        )

        await safe_edit_or_answer(callback, text, offer_view_keyboard(offer.id, offer.channel_ur
l))
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data.startswith("offer_start:"))
async def offer_start(callback: CallbackQuery):
    if not callback.from_user:
        return

    try:
        offer_id = int(callback.data.split(":")[1])

        async with async_session() as session:
            user = await get_user(session, callback.from_user.id)
            offer = await get_offer_by_id(session, offer_id)

            if not user or not offer:
                await callback.answer("\u041d\u0435 \u043d\u0430\u0439\u0434\u0435\u043d\u043e",
 show_alert=True)
                return

            success, msg = await start_offer_participation(session, user, offer)

        log_info(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id,
            offer_id=offer_id,
            success=success,
            result=msg,
        )

        text = f"\u2705 {msg}" if success else f"\u2139\ufe0f {msg}"
        if success:
            text += "\n\n\u041f\u043e\u0434\u043f\u0438\u0448\u0438\u0442\u0435\u0441\u044c \u04
3d\u0430 \u043a\u0430\u043d\u0430\u043b \u0438 \u043d\u0430\u0436\u043c\u0438\u0442\u0435 \u043f
\u0440\u043e\u0432\u0435\u0440\u0438\u0442\u044c."

        await safe_edit_or_answer(callback, text, offer_view_keyboard(offer_id, offer.channel_ur
l))
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
            offer_id=offer_id if "offer_id" in locals() else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data.startswith("offer_check:"))
async def offer_check(callback: CallbackQuery):
    if not callback.from_user:
```

```
        return

    try:
        offer_id = int(callback.data.split(":")[1])

        async with async_session() as session:
            user = await get_user(session, callback.from_user.id)
            offer = await get_offer_by_id(session, offer_id)

        if not user or not offer:
            await callback.answer("\u041d\u0435 \u043d\u0430\u0439\u0434\u0435\u043d\u043e", sho
w_alert=True)
            return

        chat_id = extract_channel_id(offer.channel_url)
        subscribed = False

        try:
            cm = await callback.bot.get_chat_member(chat_id=chat_id, user_id=callback.from_user.
id)
            subscribed = cm.status in ("member", "administrator", "creator")
        except Exception as e:
            log_warning(
                logger,
                "■■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■ ■■■■■■ get_chat_member",
                tg_id=callback.from_user.id,
                chat_id=chat_id,
                error=str(e),
            )

        async with async_session() as session:
            user = await get_user(session, callback.from_user.id)
            offer = await get_offer_by_id(session, offer_id)
            success, msg = await verify_offer_subscription(session, user, offer, subscribed)

        log_info(
            logger,
            "■■■■■■■■■ ■■■■■■■ ■■■■■■■■■■",
            tg_id=callback.from_user.id,
            offer_id=offer_id,
            subscribed=subscribed,
            success=success,
            result=msg,
        )

        await safe_edit_or_answer(
            callback,
            f"\u2705 {msg}" if success else f"\u2139\ufe0f {msg}",
            offer_view_keyboard(offer.id, offer.channel_url),
        )
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "■■■■■■■ ■■■■ ■■■■■■■■■ ■■■■■■■",
            tg_id=callback.from_user.id if callback.from_user else None,
            offer_id=offer_id if "offer_id" in locals() else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.message(F.text == BTN_GAMES)
async def games_menu(message: Message):
    log_info(
        logger,
        "■■■■■■■■ ■■■■■ ■■■■",
        tg_id=message.from_user.id if message.from_user else None,
    )
    await message.answer(
        "\U0001f3ae <b>\u0418\u0433\u0440\u044b</b>\n\n\u0412\u044b\u0431\u0435\u0440\u0438\u044
```

```
2\u0435:",
        parse_mode="HTML",
        reply_markup=games_menu_keyboard(),
    )


@router.callback_query(F.data == "game_lootbox")
async def game_lootbox(callback: CallbackQuery):
    if not callback.from_user:
        return

    try:
        async with async_session() as session:
            user = await get_user(session, callback.from_user.id)
            if not user:
                await callback.answer("/start", show_alert=True)
                return

            success, reward, msg = await play_lootbox(session, user)
            if success:
                await add_xp(session, user, XP_PER_GAME)

        log_info(
            logger,
            "■■■■ lootbox ■■■■■■■■■■",
            tg_id=callback.from_user.id,
            success=success,
            reward=reward,
            result=msg,
        )

        text = (
            f"\U0001f4e6 \u0412\u044b \u043e\u0442\u043a\u0440\u044b\u043b\u0438 \u043b\u0443\u0
442\u0431\u043e\u043a\u0441 \u0438 \u0432\u044b\u0438\u0433\u0440\u0430\u043b\u0438 <b>{reward}<
/b> \u043c\u043e\u043d\u0435\u0442!"
            if success
            else f"\u26a0\ufe0f {msg}"
        )
        await safe_edit_or_answer(callback, text, games_menu_keyboard())
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "■■■■■■ ■ lootbox",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data == "game_dice")
async def game_dice_start(callback: CallbackQuery, state: FSMContext):
    await state.set_state(GameState.waiting_dice_bet)
    await safe_edit_or_answer(
        callback,
        "\U0001f3b2 \u0412\u0432\u0435\u0434\u0438\u0442\u0435 \u0441\u0442\u0430\u0432\u043a\u0
443 \u0434\u043b\u044f \u043a\u043e\u0441\u0442\u0435\u0439 (1-50):",
    )
    await callback.answer()


@router.message(GameState.waiting_dice_bet)
async def game_dice_process(message: Message, state: FSMContext):
    if not message.from_user:
        return

    text = (message.text or "").strip()
    if not text.isdigit():
        await message.answer("\u0412\u0432\u0435\u0434\u0438\u0442\u0435 \u0447\u0438\u0441\u043
b\u043e.")
        return
```

```
    bet = int(text)

    try:
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not user:
                await message.answer("/start")
                await state.clear()
                return

            success, roll, win, msg = await play_dice(session, user, bet)
            if success:
                await add_xp(session, user, XP_PER_GAME)

        log_info(
            logger,
            "■■■■ dice ■■■■■■■■■■",
            tg_id=message.from_user.id,
            bet=bet,
            success=success,
            roll=roll,
            win=str(win),
            result=msg,
        )

        if success:
            await message.answer(
                f"\U0001f3b2 \u0412\u044b\u043f\u0430\u043b\u043e: <b>{roll}</b>\n"
                f"\U0001f4b0 \u0412\u044b\u0438\u0433\u0440\u044b\u0448: <b>{win}</b>",
                parse_mode="HTML",
                reply_markup=games_menu_keyboard(),
            )
        else:
            await message.answer(f"\u26a0\ufe0f {msg}", reply_markup=games_menu_keyboard())
    except Exception:
        log_exception(
            logger,
            "■■■■■■ ■ dice",
            tg_id=message.from_user.id if message.from_user else None,
            bet=bet,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")
    finally:
        await state.clear()


@router.callback_query(F.data == "game_coinflip")
async def game_coinflip_start(callback: CallbackQuery, state: FSMContext):
    await state.set_state(GameState.waiting_coin_bet)
    await safe_edit_or_answer(
        callback,
        "\U0001fa99 \u0412\u0432\u0435\u0434\u0438\u0442\u0435 \u0441\u0442\u0430\u0432\u043a\u0
443 \u0434\u043b\u044f \u043c\u043e\u043d\u0435\u0442\u043a\u0438 (1-50):",
    )
    await callback.answer()


@router.message(GameState.waiting_coin_bet)
async def game_coinflip_process(message: Message, state: FSMContext):
    if not message.from_user:
        return

    text = (message.text or "").strip()
    if not text.isdigit():
        await message.answer("\u0412\u0432\u0435\u0434\u0438\u0442\u0435 \u0447\u0438\u0441\u043
b\u043e.")
        return

    bet = int(text)
```

```
    try:
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not user:
                await message.answer("/start")
                await state.clear()
                return

            success, side, win, msg = await play_coinflip(session, user, bet)
            if success:
                await add_xp(session, user, XP_PER_GAME)

        log_info(
            logger,
            "[redacted] coinflip [redacted]",
            tg_id=message.from_user.id,
            bet=bet,
            success=success,
            side=side,
            win=str(win),
            result=msg,
        )

        if success:
            side_text = "\u041e\u0440\u0451\u043b" if side == "heads" else "\u0420\u0435\u0448\u
043a\u0430"
            await message.answer(
                f"\U0001fa99 {side_text}\n\U0001f4b0 \u0412\u044b\u0438\u0433\u0440\u044b\u0448:
 <b>{win}</b>",
                parse_mode="HTML",
                reply_markup=games_menu_keyboard(),
            )
        else:
            await message.answer(f"\u26a0\ufe0f {msg}", reply_markup=games_menu_keyboard())
    except Exception:
        log_exception(
            logger,
            "[redacted] coinflip",
            tg_id=message.from_user.id if message.from_user else None,
            bet=bet,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")
    finally:
        await state.clear()


@router.callback_query(F.data == "game_guess")
async def game_guess_start(callback: CallbackQuery, state: FSMContext):
    await state.set_state(GameState.waiting_guess_number)
    await safe_edit_or_answer(
        callback,
        "\U0001f522 \u0412\u0432\u0435\u0434\u0438\u0442\u0435 \u0447\u0438\u0441\u043b\u043e 1-
10:",
    )
    await callback.answer()


@router.message(GameState.waiting_guess_number)
async def game_guess_number(message: Message, state: FSMContext):
    text = (message.text or "").strip()
    if not text.isdigit():
        await message.answer("\u0412\u0432\u0435\u0434\u0438\u0442\u0435 \u0447\u0438\u0441\u043
b\u043e.")
        return

    number = int(text)
    await state.update_data(guess=number)
    await state.set_state(GameState.waiting_guess_bet)
    await message.answer("\U0001f4b0 \u0422\u0435\u043f\u0435\u0440\u044c \u0432\u0432\u0435\u04
```

```
34\u0438\u0442\u0435 \u0441\u0442\u0430\u0432\u043a\u0443 (1-50):")


@router.message(GameState.waiting_guess_bet)
async def game_guess_bet(message: Message, state: FSMContext):
    text = (message.text or "").strip()
    if not text.isdigit():
        await message.answer("\u0412\u0432\u0435\u0434\u0438\u0442\u0435 \u0447\u0438\u0441\u043
b\u043e.")
        return

    bet = int(text)
    data = await state.get_data()
    guess = int(data.get("guess", 0))

    try:
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not user:
                await message.answer("/start")
                await state.clear()
                return

            success, answer, win, msg = await play_guess(session, user, guess, bet)
            if success:
                await add_xp(session, user, XP_PER_GAME)

        log_info(
            logger,
            "■■■■■ guess ■■■■■■■■■■",
            tg_id=message.from_user.id,
            bet=bet,
            guess=guess,
            success=success,
            answer=answer,
            win=str(win),
            result=msg,
        )

        if success:
            await message.answer(
                f"\U0001f522 \u0412\u044b \u0437\u0430\u0433\u0430\u0434\u0430\u043b\u0438: <b>{
guess}</b>\n"
                f"\u041e\u0442\u0432\u0435\u0442: <b>{answer}</b>\n"
                f"\U0001f4b0 \u0412\u044b\u0438\u0433\u0440\u044b\u0448: <b>{win}</b>",
                parse_mode="HTML",
                reply_markup=games_menu_keyboard(),
            )
        else:
            await message.answer(f"\u26a0\ufe0f {msg}", reply_markup=games_menu_keyboard())
    except Exception:
        log_exception(
            logger,
            "■■■■■■■ ■ guess",
            tg_id=message.from_user.id if message.from_user else None,
            bet=bet,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")
    finally:
        await state.clear()


@router.message(F.text == BTN_TOPS)
async def tops_menu(message: Message):
    log_info(
        logger,
        "■■■■■■■■ ■■■■■ ■■■■■■",
        tg_id=message.from_user.id if message.from_user else None,
    )
    await message.answer(
```

```
        "\U0001f3c6 <b>\u0422\u043e\u043f\u044b</b>",
        parse_mode="HTML",
        reply_markup=tops_menu_keyboard(),
    )


@router.callback_query(F.data == "top_uploaders")
async def top_uploaders(callback: CallbackQuery):
    try:
        async with async_session() as session:
            rows = await get_top_uploaders(session)

        if not rows:
            text = "\u041f\u043e\u043a\u0430 \u043d\u0435\u0442 \u0434\u0430\u043d\u043d\u044b\u0
0445."
        else:
            lines = ["\U0001f3c6 <b>\u0422\u043e\u043f \u0437\u0430\u0433\u0440\u0443\u0437\u0447
7\u0438\u043a\u043e\u0432</b>\n"]
            for i, row in enumerate(rows, start=1):
                username, first_name, telegram_id, cnt = row
                name = f"@{username}" if username else (first_name or str(telegram_id))
                lines.append(f"{i}. {name} \u2014 {cnt}")
            text = "\n".join(lines)

        log_info(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
            rows_count=len(rows),
        )

        await safe_edit_or_answer(callback, text, tops_menu_keyboard())
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data == "top_viewers")
async def top_viewers(callback: CallbackQuery):
    try:
        async with async_session() as session:
            rows = await get_top_viewers(session)

        if not rows:
            text = "\u041f\u043e\u043a\u0430 \u043d\u0435\u0442 \u0434\u0430\u043d\u043d\u044b\u0
0445."
        else:
            lines = ["\U0001f440 <b>\u0422\u043e\u043f \u0437\u0440\u0438\u0442\u0435\u043b\u043
5\u0439</b>\n"]
            for i, row in enumerate(rows, start=1):
                username, first_name, telegram_id, cnt = row
                name = f"@{username}" if username else (first_name or str(telegram_id))
                lines.append(f"{i}. {name} \u2014 {cnt}")
            text = "\n".join(lines)

        log_info(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
            rows_count=len(rows),
        )

        await safe_edit_or_answer(callback, text, tops_menu_keyboard())
        await callback.answer()
    except Exception:
```

```
        log_exception(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data == "top_levels")
async def top_levels(callback: CallbackQuery):
    try:
        async with async_session() as session:
            users = await get_top_by_level(session)

        if not users:
            text = "\u041f\u043e\u043a\u0430 \u043d\u0435\u0442 \u0434\u0430\u043d\u043d\u044b\u
0445."
        else:
            lines = ["\U0001f4c8 <b>\u0422\u043e\u043f \u043f\u043e XP</b>\n"]
            for i, u in enumerate(users, start=1):
                name = f"@{u.username}" if u.username else (u.first_name or str(u.telegram_id))
                lines.append(f"{i}. {name} \u2014 lvl {u.level}, XP {u.xp}")
            text = "\n".join(lines)

        log_info(
            logger,
            "[illegible] XP",
            tg_id=callback.from_user.id if callback.from_user else None,
            users_count=len(users),
        )

        await safe_edit_or_answer(callback, text, tops_menu_keyboard())
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[illegible] XP",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data == "top_richest")
async def top_richest(callback: CallbackQuery):
    try:
        async with async_session() as session:
            users = await get_top_richest(session)

        if not users:
            text = "\u041f\u043e\u043a\u0430 \u043d\u0435\u0442 \u0434\u0430\u043d\u043d\u044b\u
0445."
        else:
            lines = ["\U0001f4b0 <b>\u0422\u043e\u043f \u0431\u043e\u0433\u0430\u0447\u0435\u043
9</b>\n"]
            for i, u in enumerate(users, start=1):
                name = f"@{u.username}" if u.username else (u.first_name or str(u.telegram_id))
                lines.append(f"{i}. {name} \u2014 {u.balance}")
            text = "\n".join(lines)

        log_info(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
            users_count=len(users),
        )

        await safe_edit_or_answer(callback, text, tops_menu_keyboard())
        await callback.answer()
    except Exception:
        log_exception(
```

```
            logger,
            "[redacted]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.message(F.text == BTN_QUESTS)
async def quests_menu(message: Message):
    if not message.from_user:
        return

    try:
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not user:
                await message.answer("/start")
                return

            quests = await ensure_daily_quests(session, user.id, user)

        log_info(
            logger,
            "[redacted]",
            tg_id=message.from_user.id,
            quests_count=len(quests),
        )

        await message.answer(
            "\U0001f3af <b>\u0415\u0436\u0435\u0434\u043d\u0435\u0432\u043d\u044b\u0435 \u043a\u0432\u0435\u0441\u0442\u044b</b>",
            parse_mode="HTML",
            reply_markup=quests_keyboard(quests),
        )
    except Exception:
        log_exception(
            logger,
            "[redacted]",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


@router.callback_query(F.data.startswith("quest_claim:"))
async def quest_claim(callback: CallbackQuery):
    if not callback.from_user:
        return

    try:
        quest_id = int(callback.data.split(":")[1])

        async with async_session() as session:
            user = await get_user(session, callback.from_user.id)
            if not user:
                await callback.answer("/start", show_alert=True)
                return

            success, msg = await claim_quest_reward(session, user, quest_id)
            quests = await ensure_daily_quests(session, user.id, user)

        log_info(
            logger,
            "[redacted]",
            tg_id=callback.from_user.id,
            quest_id=quest_id,
            success=success,
            result=msg,
        )

        text = f"\u2705 {msg}" if success else f"\u2139\ufe0f {msg}"
```

```
        await safe_edit_or_answer(callback, text, quests_keyboard(quests))
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
            quest_id=quest_id if "quest_id" in locals() else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data == "noop")
async def noop_callback(callback: CallbackQuery):
    await callback.answer()


@router.callback_query(F.data.startswith("comments:"))
async def show_comments(callback: CallbackQuery):
    try:
        video_id = int(callback.data.split(":")[1])

        async with async_session() as session:
            comments = await get_video_comments(session, video_id, limit=10)
            reactions = await get_reaction_counts(session, video_id)

        react_line = " ".join([f"{k}{v}" for k, v in reactions.items()]) if reactions else "\u04
3d\u0435\u0442"

        if not comments:
            text = (
                f"\U0001f4ac <b>\u041a\u043e\u043c\u043c\u0435\u043d\u0442\u044b \u043a #{video_
id}</b>\n\n"
                f"\u0420\u0435\u0430\u043a\u0446\u0438\u0438: {react_line}\n\n"
                "\u041f\u043e\u043a\u0430 \u043f\u0443\u0441\u0442\u043e."
            )
        else:
            lines = [
                f"\U0001f4ac <b>\u041a\u043e\u043c\u043c\u0435\u043d\u0442\u044b \u043a #{video_
id}</b>\n",
                f"\u0420\u0435\u0430\u043a\u0446\u0438\u0438: {react_line}\n"
            ]
            for c in comments:
                lines.append(f"<b>{c['author']}</b>: {c['text']}")
            text = "\n".join(lines)

        log_info(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
            video_id=video_id,
            comments_count=len(comments),
            reactions_count=len(reactions),
        )

        await callback.message.answer(text, parse_mode="HTML")
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data.startswith("comment_add:"))
async def add_comment_start(callback: CallbackQuery, state: FSMContext):
    try:
        video_id = int(callback.data.split(":")[1])
```

```
        await state.update_data(comment_video_id=video_id)
        await state.set_state(CommentState.waiting_comment_text)

        log_info(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
            video_id=video_id,
        )

        await callback.message.answer(
            f"\u270d\ufe0f \u041d\u0430\u043f\u0438\u0448\u0438\u0442\u0435 \u043a\u043e\u043c\u
043c\u0435\u043d\u0442 \u0434\u043b\u044f #{video_id}\n/start \u2014 \u043e\u0442\u043c\u0435\u043
43d\u0430"
        )
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.message(CommentState.waiting_comment_text)
async def add_comment_process(message: Message, state: FSMContext):
    if not message.from_user:
        return

    text = (message.text or "").strip()
    if len(text) < 1:
        await message.answer("\u041f\u0443\u0441\u0442\u043e\u0439 \u043a\u043e\u043c\u043c\u0435\u043
5\u043d\u0442.")
        return

    try:
        data = await state.get_data()
        video_id = int(data["comment_video_id"])

        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not user:
                await message.answer("/start")
                await state.clear()
                return

            await add_comment(session, user.id, video_id, text)
            await add_xp(session, user, XP_PER_COMMENT)
            await increment_quest(session, user.id, "comment")

        log_info(
            logger,
            "[illegible]",
            tg_id=message.from_user.id,
            video_id=video_id,
            text_length=len(text),
        )

        await message.answer("\u2705 \u041a\u043e\u043c\u043c\u0435\u043d\u0442 \u0434\u043e\u04
31\u0430\u0432\u043b\u0435\u043d.")
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")
    finally:
        await state.clear()
```

```
@router.callback_query(F.data.startswith("react_menu:"))
async def react_menu(callback: CallbackQuery):
    try:
        video_id = int(callback.data.split(":")[1])

        log_info(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
            video_id=video_id,
        )

        await callback.message.answer(
            "\u2764\ufe0f <b>\u0420\u0435\u0430\u043a\u0446\u0438\u0438</b>",
            parse_mode="HTML",
            reply_markup=reaction_menu_keyboard(video_id),
        )
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.callback_query(F.data.startswith("react:"))
async def react_process(callback: CallbackQuery):
    if not callback.from_user:
        return

    try:
        _, video_id, reaction = callback.data.split(":", 2)
        if reaction not in REACTION_TYPES:
            await callback.answer("\u041d\u0435\u043b\u044c\u0437\u044f", show_alert=True)
            return

        async with async_session() as session:
            user = await get_user(session, callback.from_user.id)
            if not user:
                await callback.answer("/start", show_alert=True)
                return

            await add_reaction(session, user.id, int(video_id), reaction)
            await add_xp(session, user, XP_PER_REACTION)
            await increment_quest(session, user.id, "react")

        log_info(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id,
            video_id=int(video_id),
            reaction=reaction,
        )

        await callback.answer(f"{reaction} \u0441\u043e\u0445\u0440\u0430\u043d\u0435\u043d\u043
e")
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.message(F.text == BTN_UPLOAD)
```

```
async def upload_prompt(message: Message):
    log_info(
        logger,
        "[illegible]",
        tg_id=message.from_user.id if message.from_user else None,
    )
    await message.answer(
        "\u041e\u0442\u043f\u0440\u0430\u0432\u044c\u0442\u0435 \u0432\u0438\u0434\u0435\u043e,
\u043a\u0440\u0443\u0436\u043e\u043a \u0438\u043b\u0438 \u0444\u043e\u0442\u043e.\n\n"
        "\u041d\u0430\u0433\u0440\u0430\u0434\u0430: \u0432\u0438\u0434\u0435\u043e/\u043a\u0440
\u0443\u0436\u043e\u043a <b>0.5</b>, \u0444\u043e\u0442\u043e <b>0.1</b>",
        parse_mode="HTML",
    )


@router.message(F.video)
async def handle_video(message: Message):
    if not message.from_user or not message.video:
        return

    await _upload_video(
        message,
        message.video.file_id,
        message.video.file_unique_id,
        message.video.duration,
        message.video.file_size,
    )


@router.message(F.video_note)
async def handle_vnote(message: Message):
    if not message.from_user or not message.video_note:
        return

    await _upload_video(
        message,
        message.video_note.file_id,
        message.video_note.file_unique_id,
        message.video_note.duration,
        message.video_note.file_size,
    )


async def _upload_video(message, file_id, file_unique_id, duration, file_size):
    try:
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not user or not user.agreed_to_rules:
                await message.answer("/start")
                return

            video = await save_video(session, user, file_id, file_unique_id, duration, file_size
)
            if video is None:
                log_warning(
                    logger,
                    "[illegible]",
                    tg_id=message.from_user.id,
                    file_unique_id=file_unique_id,
                )
                await message.answer("\u26a0\ufe0f \u0414\u0443\u0431\u043b\u0438\u043a\u0430\u0
442.")
                return

            await add_xp(session, user, XP_PER_UPLOAD)
            await increment_quest(session, user.id, "upload")

        log_info(
            logger,
            "[illegible]",
```

```
            tg_id=message.from_user.id,
            video_id=video.id,
            duration=duration,
            file_size=file_size,
        )

        await message.answer(
            "\u2705 \u041d\u0430 \u043c\u043e\u0434\u0435\u0440\u0430\u0446\u0438\u0438. \u041d\
u0430\u0433\u0440\u0430\u0434\u0430: <b>0.5</b>",
            parse_mode="HTML",
        )
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


@router.message(F.photo)
async def handle_photo(message: Message):
    if not message.from_user or not message.photo:
        return

    try:
        largest = message.photo[-1]

        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not user or not user.agreed_to_rules:
                await message.answer("/start")
                return

            photo = await save_photo(session, user, largest.file_id, largest.file_unique_id, lar
gest.file_size)
            if photo is None:
                log_warning(
                    logger,
                    "[illegible]",
                    tg_id=message.from_user.id,
                    file_unique_id=largest.file_unique_id,
                )
                await message.answer("\u26a0\ufe0f \u0414\u0443\u0431\u043b\u0438\u043a\u0430\u0
442.")
                return

            await add_xp(session, user, XP_PER_UPLOAD)
            await increment_quest(session, user.id, "upload")

        log_info(
            logger,
            "[illegible]",
            tg_id=message.from_user.id,
            photo_id=photo.id,
            file_size=largest.file_size,
        )

        await message.answer(
            "\u2705 \u041d\u0430 \u043c\u043e\u0434\u0435\u0440\u0430\u0446\u0438\u0438. \u041d\
u0430\u0433\u0440\u0430\u0434\u0430: <b>0.1</b>",
            parse_mode="HTML",
        )
    except Exception:
        log_exception(
            logger,
            "[illegible]",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")
```

```
@router.message(F.text == BTN_WATCH)
async def watch_menu(message: Message):
    log_info(
        logger,
        "[redacted]",
        tg_id=message.from_user.id if message.from_user else None,
    )
    await message.answer("\u0412\u044b\u0431\u0435\u0440\u0438\u0442\u0435:", reply_markup=watch
_choice_keyboard())


@router.callback_query(F.data == "watch_video_content")
async def cb_wv(callback: CallbackQuery):
    if callback.from_user:
        await _send_video(callback.message, callback.from_user.id)
    await callback.answer()


@router.callback_query(F.data == "watch_next")
async def cb_wn(callback: CallbackQuery):
    if callback.from_user:
        await _send_video(callback.message, callback.from_user.id)
    await callback.answer()


@router.callback_query(F.data == "watch_photo_content")
async def cb_wp(callback: CallbackQuery):
    if callback.from_user:
        await _send_photo(callback.message, callback.from_user.id)
    await callback.answer()


@router.callback_query(F.data == "watch_next_photo")
async def cb_wnp(callback: CallbackQuery):
    if callback.from_user:
        await _send_photo(callback.message, callback.from_user.id)
    await callback.answer()


async def _send_video(message, telegram_id):
    try:
        async with async_session() as session:
            user = await get_user(session, telegram_id)
            if not user:
                await message.answer("/start")
                return

            if user.balance < Decimal(str(WATCH_COST)):
                await message.answer("\u274c \u041d\u0435\u0434\u043e\u0441\u0442\u0430\u0442\u0
43e\u0447\u043d\u043e \u043c\u043e\u043d\u0435\u0442.")
                return

            video = await get_random_video_for_user(session, user)
            if not video:
                await message.answer("\U0001f4ed \u041d\u0435\u0442 \u0432\u0438\u0434\u0435\u04
3e.")
                return

            charged = await record_view_and_charge(session, user, video)
            if not charged:
                await message.answer("\u274c \u041e\u0448\u0438\u0431\u043a\u0430.")
                return

            await add_xp(session, user, XP_PER_WATCH)
            await increment_quest(session, user.id, "watch")
            await session.refresh(user)

            fid = video.telegram_file_id
```

```
            vid = video.id
            bal = user.balance

        log_info(
            logger,
            "[redacted]",
            tg_id=telegram_id,
            video_id=vid,
            balance_after=str(bal),
        )

        await message.answer_video(
            video=fid,
            caption=f"\U0001f4b0 -1. \u0411\u0430\u043b\u0430\u043d\u0441: <b>{bal}</b>",
            parse_mode="HTML",
            reply_markup=video_rating_keyboard(vid),
        )
    except Exception:
        log_exception(
            logger,
            "[redacted]",
            tg_id=telegram_id,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


async def _send_photo(message, telegram_id):
    try:
        async with async_session() as session:
            user = await get_user(session, telegram_id)
            if not user:
                await message.answer("/start")
                return

            if not is_vip(user):
                vc = await count_photo_views_last_4h(session, user.id)
                if vc >= FREE_PHOTO_LIMIT_PER_4H:
                    await message.answer(f"\u26d4 \u041b\u0438\u043c\u0438\u0442 {FREE_PHOTO_LIM
IT_PER_4H} \u0444\u043e\u0442\u043e/4\u0447.")
                    return
            else:
                vc = 0

            photo = await get_random_photo_for_user(session, user)
            if not photo:
                await message.answer("\U0001f4ed \u041d\u0435\u0442 \u0444\u043e\u0442\u043e.")
                return

            await record_photo_view(session, user, photo)
            rem = "\u221e" if is_vip(user) else str(FREE_PHOTO_LIMIT_PER_4H - (vc + 1))
            fid = photo.telegram_file_id
            pid = photo.id

        log_info(
            logger,
            "[redacted]",
            tg_id=telegram_id,
            photo_id=pid,
            remaining=rem,
        )

        await message.answer_photo(
            photo=fid,
            caption=f"\U0001f5bc \u041e\u0441\u0442\u0430\u043b\u043e\u0441\u044c: <b>{rem}</b>"
,
            parse_mode="HTML",
            reply_markup=photo_actions_keyboard(),
        )
    except Exception:
        log_exception(
```

```
            logger,
            "[redacted]",
            tg_id=telegram_id,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


@router.callback_query(F.data.startswith("rate:"))
async def cb_rate(callback: CallbackQuery):
    if not callback.from_user:
        return

    try:
        parts = callback.data.split(":")
        vid = int(parts[1])
        r = int(parts[2])

        async with async_session() as session:
            user = await get_user(session, callback.from_user.id)
            if user:
                await rate_video(session, user.id, vid, r)
                await add_xp(session, user, XP_PER_RATING)
                await increment_quest(session, user.id, "rate")

        log_info(
            logger,
            "[redacted]",
            tg_id=callback.from_user.id,
            video_id=vid,
            rating=r,
        )

        await callback.answer("\u041e\u0446\u0435\u043d\u043a\u0430 \u0441\u043e\u0445\u0440\u0430
30\u043d\u0435\u043d\u0430")
    except Exception:
        log_exception(
            logger,
            "[redacted]",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.message(F.text == BTN_ADMIN)
async def open_admin(message: Message):
    if not message.from_user:
        return

    try:
        async with async_session() as session:
            user = await get_user(session, message.from_user.id)
            if not is_any_admin(message.from_user.id, user):
                await message.answer("\u26d4")
                return

            sa = is_super_admin(message.from_user.id)

        log_info(
            logger,
            "[redacted]",
            tg_id=message.from_user.id,
            is_super_admin=sa,
        )

        await message.answer(
            "\U0001f6e0 <b>\u0410\u0434\u043c\u0438\u043d</b>",
            parse_mode="HTML",
            reply_markup=admin_center_keyboard(is_super_admin=sa),
        )
    except Exception:
```

```
        log_exception(
            logger,
            "[illegible]",
            tg_id=message.from_user.id if message.from_user else None,
        )
        await message.answer("\u041e\u0448\u0438\u0431\u043a\u0430.")


@router.callback_query(F.data == "admin_center")
async def cb_admin_center(callback: CallbackQuery):
    if not callback.from_user:
        return

    try:
        async with async_session() as session:
            user = await get_user(session, callback.from_user.id)
            if not is_any_admin(callback.from_user.id, user):
                await callback.answer("\u26d4", show_alert=True)
                return

            sa = is_super_admin(callback.from_user.id)

        log_info(
            logger,
            "[illegible] admin_center callback",
            tg_id=callback.from_user.id,
            is_super_admin=sa,
        )

        await safe_edit_or_answer(
            callback,
            "\U0001f6e0 <b>\u0410\u0434\u043c\u0438\u043d</b>",
            admin_center_keyboard(is_super_admin=sa),
        )
        await callback.answer()
    except Exception:
        log_exception(
            logger,
            "[illegible] admin_center",
            tg_id=callback.from_user.id if callback.from_user else None,
        )
        await callback.answer("\u041e\u0448\u0438\u0431\u043a\u0430", show_alert=True)


@router.message()
async def fallback_unknown_message(message: Message):
    if not message.from_user:
        return

    log_warning(
        logger,
        "[illegible]",
        tg_id=message.from_user.id,
        text=message.text,
    )

    await message.answer(
        "\u2139\ufe0f \u042f \u043d\u0435 \u043f\u043e\u043d\u044f\u043b \u044d\u0442\u0443 \u043
3a\u043e\u043c\u0430\u043d\u0434\u0443.\n"
        "\u0418\u0441\u043f\u043e\u043b\u044c\u0437\u0443\u0439\u0442\u0435 /start \u0438\u043b\
u0438 \u043a\u043d\u043e\u043f\u043a\u0438 \u043c\u0435\u043d\u044e."
    )
```